PROJECT REPORT

Placemate A Software-Engineering–Driven Placement Opportunity Aggregator

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY
IN

COMPUTER SCIENCE & ENGINEERING - Specialization

Submitted by

NAME1- Ayush Goel	**University Roll No- 2218560**
NAME2- Anshika Kandari	**University Roll No- 2218397**
NAME2- Saloni Bhandari	**University Roll No- 2219530**
NAME2- Osheen Rudola	**University Roll No- 2219201**

Under the guidance of

Ms Shraddha Kaparwan

Assistant Professor,Graphic Era Hill University

Project Group No: 217

Department of Computer Science and Engineering
Graphic Era Hill University

May, 2026

Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project progress report entitled **"Placemate A Software-Engineering–Driven Placement Opportunity Aggregator "** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering **- Specialization** in the Department of Computer Science and Engineering of the Graphic Era Hill University, Dehradun shall be carried out by the undersigned under the supervision of Ms Shraddha Kaparwan, Assistant Professor, Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun.

NAME1- Ayush Goel	**University Roll No- 2218560**
NAME2- Anshika Kandari	**University Roll No- 2218397**
NAME2- Saloni Bhandari	**University Roll No- 2219530**
NAME2- Osheen Rudola	**University Roll No- 2219201**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our project guide Ms Shraddha Kaparwan ,Assistant Professor Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun, for their invaluable guidance, constant support, and encouragement throughout the development of this project.
We are grateful to the Head of the Department and all faculty members of the Department of Computer Science and Engineering for providing us with the necessary resources and infrastructure.
We would also like to thank our family and friends for their unwavering support and motivation during the course of this project.

Name of student 1 : Ayush Goel
Roll no. 2218560

Name of student 2 :Anshika Kandari
Roll no. 2218397

Name of student 2 : Saloni Bhandari
Roll no. 2219530

Name of student 4 : Osheen Rudola
Roll no. 22919201

ABSTRACT

The modern placement ecosystem faces multifaceted challenges rooted in fragmentation, data inaccessibility, and the absence of an intelligent centralized coordination mechanism. Students seeking employment opportunities are often required to navigate multiple disconnected portals, manually track application statuses, and independently manage company-wise eligibility requirements — a process that is not only time-consuming but inherently prone to oversight and error. Simultaneously, placement coordinators and recruiters struggle with manual workflows, redundant administrative tasks, and limited analytical visibility into recruitment pipelines.

PlaceMate is a full-stack, software-engineering–driven Placement Opportunity Aggregator designed and developed to address these systemic inefficiencies. Built on the MERN (MongoDB, Express.js, React.js, Node.js) technology stack, PlaceMate provides a unified digital platform that consolidates job listings, student profiles, application tracking, recruiter dashboards, and placement coordinator analytics under a single, coherent architecture.

The system implements a multi-role architecture with distinct interfaces and access controls for students, recruiters, and administrators. Students can create detailed profiles, upload resumes, discover and filter job opportunities based on eligibility criteria, and track their application pipeline in real time. Recruiters can post structured job listings, define eligibility requirements, review applicant pools, and manage the shortlisting workflow. Administrators — typically placement coordinators — gain a bird's-eye view of all ongoing placement activities, with access to aggregated analytics and reporting tools.

PlaceMate employs RESTful API architecture for seamless frontend–backend communication, JWT-based stateless authentication for secure session management, and MongoDB Atlas for scalable cloud-hosted data persistence. The React.js frontend leverages component-based design for modularity and reusability, while the Express.js backend enforces structured business logic through well-defined middleware pipelines.

Evaluation of PlaceMate demonstrates significant improvements over traditional placement systems across key metrics: time-to-apply reduction, improved application tracking accuracy, and reduced administrative overhead. The system is architected with extensibility in mind, with a modular codebase that supports future integration of AI-driven job recommendation engines, interview scheduling systems, and analytics dashboards powered by machine learning.

This report presents a comprehensive account of the system design, implementation methodology, technology decisions, and performance evaluation of PlaceMate. It also

discusses limitations of existing systems, the theoretical underpinnings of the proposed architecture, and future directions for research and development in automated placement systems.

Keywords: *Placement Aggregator, MERN Stack, RESTful API, JWT Authentication, MongoDB, React.js, Job Portal, Recruitment Automation, Multi-Role System, Full-Stack Development.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT
ABSTRACT
LIST OF TABLES
LIST OF FIGURES
ABBREVIATIONS
NOTATIONS

CHAPTER 1: INTRODUCTION AND MOTIVATION
1.1 Background of Placement Systems
1.2 Problem Domain
1.3 Existing System and Limitations
1.4 Proposed System (PlaceMate)
1.5 Motivation for the Project

CHAPTER 2: BACKGROUND STUDY
2.1 Traditional Placement Systems
2.2 Job Portals and Their Limitations
2.3 Need for Centralized Platform
2.4 Role of Automation in Placement Systems0
2.5 Gap Analysis and Our Contribution

CHAPTER 3: SOFTWARE AND HARDWARE REQUIREMENTS
3.1 Hardware Requirements
3.2 Software Requirements
3.2.1 Programming Languages
3.2.2 Frameworks and Libraries
3.2.3 Cloud and Hosting Platforms

3.3 Software Requirements Specification (SRS)
3.3.1 Functional Requirements
3.3.2 Non-Functional Requirements

3.4 Use Case Diagram
3.5 System Architecture

CHAPTER 4: PROJECT METHODOLOGY / DESIGN
4.1 Introduction to Methodology
4.2 Phase 1 –Technology Stack Overview

4.2.1 MongoDB-Data Layer Design
4.2.2 Express.Js- API Design
4.3 React.js -Frontend Architecture

4.4 Authentication and Authorization Flow

4.5 Database Integration and Mongoose ODM

4.5.1 ER Diagram and Data Relationships

4.5.2 File Upload and Media Management

4.5.3 Deployment Architecture

CHAPTER 5: IMPLEMENTATION AND RESULTS
5.1 Module-wise Implementation
5.2 Frontend Implementation
5.3 Backend Implementation
5.4 Database Integration
5.5 System Outputs and Screenshots
5.6 Results and Performance Analysis

CHAPTER 6: CONCLUSION AND FUTURE SCOPE
6.1 Conclusion
6.2 Applications
6.3 Future Scope

REFERENCES

APPENDIX CODE

LIST OF TABLES

Table 1.1 – Background of Placement Systems
Table 1.2 – Problem Domain
Table 1.3 – Existing System and Limitation
Table 1.4 – Proposed System (Placemate)
Table 1.5 – Motivation for the Project
Table 2.1 – Traditional Placement System
Table 2.2 – Job Portal and Their Limitations
Table 2.3 – Need For Centralized Platform
Table 2.4 – Role of Automation in Placement Systems
Table 2.5 – Gap Analysis and Our Contribution
Table 3.1 – Hardware Requirements Specification
Table 3.2 – Software Requirements
Table 3.3 – Software Requirements Specification
Table 3.4 – User Case Diagram
Table 3.5 – System Architecture
Table 4.1 – Introduction to Methodology
Table 4.2 – Technology Stack Summary
Table 4.3 – React Frontend Architecture
Table 4.4 – Authentication and Authorization Flow
Table 4.5 – Database Integration and Mongoose ODM
Table 5.1 – Module-wise Implementation Timeline
Table 5.2 – Frontend Implementation
Table 5.3 – Backend Implementation
Table 5.4 – Database Integration
Table 5.5 – Sysem Output and Screenshots
Table 6.1 – Conclusion
Table 6.2 – Application
Table 6.3 – Future Scope

LIST OF FIGURES

Figure 1.1 – Use Case Diagram – PlaceMate System
Figure 1.2 – System Architecture Diagram
Figure 1.3 – Frontend Tree
Figure 1.4 – Backend Component
Figure 1.5 – Database Component
Figure 1.6 – Login / Registration Page – Screenshot
Figure 1.7 – Student Dashboard – Screenshot
Figure 1.8 – Job Listing Page – Screenshot
Figure 1.9 – Recruiter Dashboard – Screenshot
Figure 2.0 – Admin Analytics Dashboard – Screenshot

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
MVC	Model-View-Controller
NoSQL	Not Only Structured Query Language
npm	Node Package Manager
REST	Representational State Transfer
SPA	Single Page Application
SRS	Software Requirements Specification
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

NOTATIONS

Notation	Description
{ }	Denotes a JSON object or set
[ ]	Denotes a JSON array or list
→	Maps to / results in
≥	Greater than or equal to
≤	Less than or equal to
∈	Belongs to a set
n	Number of records / users
O(n)	Linear time complexity
req	HTTP Request object (Express.js)
res	HTTP Response object (Express.js)
_id	MongoDB Document Identifier (ObjectId)
T(ms)	Response time in milliseconds

CHAPTER 1: INTRODUCTION AND MOTIVATION

1.1 Background of Placement Systems

The landscape of campus recruitment has undergone a significant transformation over the past two decades. Traditionally, campus placements were orchestrated exclusively through the Training and Placement (T&P) cells of academic institutions — departments responsible for establishing industry contacts, organizing on-campus drives, and facilitating student-recruiter interactions. This traditional model, while functional in its early context, was inherently limited in scale, transparency, and technological sophistication.

In the pre-digital era, placement processes relied heavily on physical notice boards, printed brochures, and face-to-face interactions. Students depended on word-of-mouth and institutional announcements to stay informed about upcoming recruitment drives. Recruiters, on the other hand, depended entirely on the institutional intermediary — the T&P cell — to source and filter candidates, without any direct access to student databases or performance metrics.

The emergence of the internet in the late 1990s catalysed the first wave of digital transformation in recruitment, giving rise to dedicated job portals such as Naukri, Monster, and LinkedIn. These platforms broke the geographic barrier, enabling recruiters to post job listings to a national audience and allowing job seekers to create digital profiles accessible to multiple employers. However, these platforms were designed as general-purpose job boards and were not calibrated for the unique requirements of campus placements — where batch eligibility, CGPA filters, department-specific criteria, and coordinated interview scheduling are paramount.

The second wave of transformation was driven by mobile computing and cloud technologies, which democratized access to information and introduced real-time interactivity. Yet, for the campus placement domain, this wave brought with it a paradox: as the number of online platforms grew, so did the fragmentation. Students were now required to maintain profiles on multiple platforms, apply to jobs through disparate portals, and manually track application statuses — a cognitive overhead that disproportionately affected first-generation job seekers with limited digital literacy.

It is within this context that PlaceMate emerges as a solution — a purpose-built, full-stack placement opportunity aggregator that brings together job discovery, profile management, application tracking, and administrative oversight into a single, coherent platform.

1.2 Problem Domain

The problem domain of PlaceMate sits at the intersection of software engineering, human-computer interaction, and organizational workflow automation. At its core, the project addresses four interconnected challenges that characterize the current state of campus placement systems:

First, information fragmentation is a pervasive issue. Job listings relevant to a particular batch of students are scattered across institutional portals, LinkedIn, Naukri, company career pages, and WhatsApp group messages. There is no canonical, authoritative source of truth for ongoing placement opportunities. This fragmentation leads to missed opportunities, duplicated applications, and an asymmetry of information between well-connected students and their peers.

Second, the absence of structured eligibility enforcement results in a mismatch between applicants and job requirements. Without a system that enforces CGPA thresholds, branch-specific eligibility, and year-of-passing criteria at the application stage, both students and recruiters waste significant time processing ineligible applications.

Third, application tracking remains largely manual. Students maintain personal spreadsheets to track which companies they have applied to, the status of their applications, and upcoming interview dates. This manual tracking is error-prone and offers no real-time updates when application statuses change on the recruiter's end.

Fourth, placement coordinators lack data-driven insights. The T&P cell currently operates without access to real-time analytics on placement rates, popular job roles, department-wise performance, or recruiter engagement. This limits their ability to make informed decisions about outreach, training programs, and strategic partnerships.

1.3 Existing System and Limitations

Several existing platforms partially address the challenges outlined above. A critical examination of these systems reveals inherent limitations that PlaceMate is designed to overcome.

LinkedIn is the world's largest professional networking platform and a widely-used job search tool. However, its general-purpose nature means it lacks campus-specific features such as batch management, bulk application processing, and institutional administrator access. LinkedIn's algorithm is optimized for professional networking rather than structured campus recruitment, making it unsuitable as a primary placement management system.

Naukri.com and Monster.com are among India's most prominent job portals. While they offer extensive job listings and resume-builder tools, they do not support multi-role institutional access, batch-level filtering, or integrated analytics for placement coordinators. Furthermore, their platforms are not optimized for fresh graduates without prior work experience, often deprioritizing fresher profiles in search rankings.

Many institutions use custom-built Excel-based or ERP-integrated systems for managing placement data. These systems suffer from poor scalability, limited accessibility (typically confined to on-campus networks), and lack of real-time synchronization. Manual data entry introduces errors, and the absence of a standardized interface creates training overhead for new administrators.

Some institutions have deployed proprietary placement management software, but these tend to be expensive, inflexible to customization, and poorly integrated with modern web technologies. They often lack mobile responsiveness and offer limited API extensibility for future integrations.

In summary, the existing ecosystem is characterized by: general-purpose platforms not optimized for campus placement; proprietary systems that are expensive and inflexible; manual workflows prone to error and information loss; and a lack of real-time, data-driven decision support for placement coordinators.

1.4 Proposed System (PlaceMate)

PlaceMate is proposed as a comprehensive, open-architecture, full-stack web application that serves as a centralized placement opportunity aggregator. The system is designed around three primary stakeholder roles: students, recruiters, and placement administrators. Each role is accorded a tailored interface, distinct access controls, and role-specific functionalities.

For students, PlaceMate provides a unified dashboard that aggregates all relevant job opportunities matching their profile. The system automatically filters job listings based on the student's CGPA, department, and graduation year, presenting only eligible opportunities. Students can submit applications with a single click, attach their resume, and track the real-time status of each application through a Kanban-style pipeline view.

For recruiters, PlaceMate offers a structured job posting interface where they can define role requirements, set eligibility criteria, and manage applicant pools. Recruiters receive automated notifications when new eligible applications are submitted and can update application statuses directly through the platform, triggering real-time notifications to the respective students.

For placement administrators, PlaceMate provides an analytics dashboard that offers insights into placement rates, offer statistics, department-wise performance, and recruiter engagement. Administrators can manage student and recruiter accounts, approve or reject job postings, and generate exportable placement reports.

The system is built on the MERN stack — MongoDB for data persistence, Express.js for RESTful API development, React.js for the client-side interface, and Node.js as the runtime environment. JWT-based authentication ensures secure, stateless session management, while Cloudinary integration enables scalable resume and profile picture storage.

1.5 Motivation for the Project

The motivation for PlaceMate stems from direct observation of the inefficiencies that afflict campus placement processes at technical institutions. As students navigating our own placement season, the project team experienced firsthand the cognitive overhead of managing applications across multiple platforms, the anxiety of missing deadlines buried in WhatsApp notification threads, and the frustration of applying to roles for which eligibility criteria were opaque or inconsistently enforced.

Beyond personal experience, the broader societal imperative is compelling. Campus placements are a critical inflection point in the career trajectories of millions of engineering graduates in India. Suboptimal placement systems disproportionately disadvantage students from underrepresented backgrounds who lack strong alumni networks or digital literacy. A well-designed, accessible, and intelligent placement platform can serve as an equalizer — surfacing relevant opportunities to all eligible students simultaneously, irrespective of their social capital.

Furthermore, the software engineering dimension of this project presented an intellectually stimulating challenge: designing a multi-role, real-time, cloud-connected web application that adheres to industry-standard architectural patterns. The project provided an opportunity to apply theoretical concepts from software engineering, database management, and web development in an integrated, practical context.

The motivation is thus threefold: to solve a real, well-defined problem with a tangible social impact; to demonstrate technical proficiency in full-stack MERN development; and to produce a platform that is genuinely extensible and deployable at institutional scale.

CHAPTER 2: BACKGROUND STUDY

2.1 Traditional Placement Systems

Traditional placement systems in academic institutions were fundamentally manual, paper-based processes coordinated by the Training and Placement (T&P) cell. The typical workflow involved the following sequential stages: company registration and invitation, student database preparation, eligibility screening, written test administration, interview scheduling, offer letter collection, and final placement recording.

Each of these stages relied on physical documentation and human intermediaries. Company representatives would visit the campus with printed job descriptions. T&P coordinators would manually cross-reference student CGPA records — maintained in department-specific Excel files — against the company's eligibility criteria. Eligible students would be informed via notice boards or class representatives. Written tests were conducted in physical examination halls with paper answer sheets. Interview schedules were printed and distributed. Offer letters were physically collected from company representatives and recorded in institutional placement registers.

This system, while operationally familiar, suffered from significant structural weaknesses. The manual eligibility screening process was time-consuming and error-prone, particularly when processing large student batches. Information asymmetry was endemic: students with connections to senior batches or T&P cell representatives were often better informed about upcoming drives than their peers. The absence of a centralized database meant that historical placement data was siloed within individual T&P cells, preventing cross-institutional benchmarking or longitudinal analysis.

From a software engineering perspective, traditional placement systems violated virtually every principle of good system design: they lacked modularity (each T&P cell operated as an isolated, incompatible island), scalability (processes that worked for a batch of 100 students broke down for batches of 1,000), and auditability (there was no systematic logging of system activities or changes to records).

2.2 Job Portals and Their Limitations

The first generation of online job portals, including Naukri.com, Shine.com, and Freshersworld.com, represented a significant improvement over manual systems by digitizing job listings and enabling electronic resume submission. These platforms introduced keyword-based search, structured job descriptions, and email notification systems. They democratized access to job opportunities by making listings visible to anyone with internet access, regardless of geographic location.

However, general-purpose job portals were not designed with campus placement in mind. They lack several critical features that are specific to the institutional recruitment context. First, they do not support batch-level management: there is no mechanism on Naukri or LinkedIn to define a specific graduation batch (e.g., B.Tech 2025) and restrict visibility of a job listing to that batch. Second, they do not enforce institution-specific eligibility criteria: CGPA thresholds, active backlog policies, and department restrictions are academic parameters that general portals are not equipped to handle.

Second-generation platforms, such as LinkedIn, added sophisticated networking features, recommendation algorithms, and skill endorsements. LinkedIn's Jobs feature further blurred the boundary between professional networking and job searching. However, LinkedIn's algorithm favors profiles with extensive work experience, endorsements, and network connections — criteria that disadvantage fresh graduates who are applying for their first professional role.

Campus-specific platforms such as CampusRecruiter, Superset (now CareerNet), and Unstop have emerged in recent years to address some of these gaps. These platforms support college-specific accounts, batch management, and campus drive scheduling. However, they are proprietary SaaS products with subscription-based pricing models that are inaccessible to smaller institutions with limited budgets. Moreover, they offer limited customization and do not provide source code access for institutions wishing to integrate placement data into their existing ERP systems.

2.3 Need for Centralized Platform

The inadequacy of both traditional manual systems and general-purpose job portals points to a clear and well-defined need: a centralized, institution-deployable, role-aware placement

management platform. Such a platform must fulfil several non-negotiable requirements that emerge from the analysis of existing systems.

Centralization of information is the most fundamental requirement. All job listings, student profiles, application records, and placement analytics must be stored in and accessible from a single, authoritative platform. This eliminates the fragmentation that characterizes the current ecosystem and ensures that all stakeholders — students, recruiters, and administrators — are operating on the same information.

Role-based access control is equally critical. The platform must distinguish between student, recruiter, and administrator roles, providing each with a tailored interface and appropriate data access permissions. A student should not be able to view other students' application statuses; a recruiter should not be able to modify student academic records; an administrator should have read access to all data but should not be able to submit applications on behalf of students.

Real-time synchronization ensures that changes made by one stakeholder are immediately reflected in the interfaces of other relevant stakeholders. When a recruiter updates an application status from 'Under Review' to 'Shortlisted', the affected student should receive an immediate notification. This real-time feedback loop reduces uncertainty and enables timely action.

Institutional customizability allows the platform to be deployed with institution-specific configurations: custom eligibility criteria, branding, and reporting templates. This flexibility is essential for adoption across institutions with diverse placement policies.

2.4 Role of Automation in Placement Systems

Automation is a central design principle of PlaceMate. Within the context of placement systems, automation serves multiple purposes: reducing administrative overhead, enforcing consistency in eligibility screening, enabling real-time notifications, and generating analytics from accumulated data.

Eligibility automation is perhaps the most impactful application. When a recruiter defines a job with a minimum CGPA of 7.5, restricted to Computer Science and Information Technology departments, the system automatically evaluates each student's profile against these criteria. Only eligible students see the job listing in their dashboard. This automated eligibility filtering

eliminates a step that, in manual systems, requires a T&P coordinator to cross-reference hundreds of student records — a process that takes hours and is subject to human error.

Notification automation reduces the latency between an event and the relevant stakeholder's awareness of it. When a student submits an application, the recruiter receives an automated email and in-app notification. When a recruiter schedules an interview, the student is automatically notified with the date, time, and venue. This automated communication chain replaces the manual phone calls, emails, and notice board updates that characterize traditional placement communication.

Analytics automation transforms raw placement data into actionable insights without requiring manual data compilation. Placement coordinators can access real-time dashboards showing the number of registered companies, total offers made, department-wise placement rates, and average package statistics. These metrics are computed automatically from the underlying database and updated in real time as new data is entered.

2.5 Gap Analysis and Our Contribution

A systematic gap analysis of existing placement systems reveals several areas where current solutions fall short of the ideal. The following table summarises the key gaps identified and the corresponding contributions of PlaceMate.

Gap in Existing Systems	PlaceMate Contribution
No single platform for all placement activities	Unified MERN stack platform with role-based dashboards
Manual eligibility filtering by coordinators	Automated server-side eligibility enforcement at application stage
No real-time application status tracking	Kanban-style pipeline view with real-time status updates
Limited analytics for placement coordinators	Admin analytics dashboard with placement KPIs and reports
High cost of proprietary campus placement SaaS	Open-source, self-hostable architecture with zero licensing cost
Poor mobile experience on legacy systems	React.js SPA with responsive design for all device sizes
No structured recruiter-student communication	In-app notification system with email fallback

The primary academic contribution of this project lies in demonstrating that a software-engineering–driven approach — applying principles such as separation of concerns, RESTful

API design, component-based UI architecture, and JWT-based stateless authentication — can produce a placement management system that is simultaneously more functional, more scalable, and more cost-effective than existing alternatives. The project also contributes a reusable architectural template for similar multi-role, real-time web applications in the educational technology domain.

CHAPTER 3: SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Hardware Requirements

The hardware requirements for PlaceMate are specified for two contexts: the development environment used during construction of the system, and the deployment environment required for production hosting. Given that the system is a cloud-hosted web application, the end-user hardware requirements are minimal — any device capable of running a modern web browser with internet connectivity is sufficient.

Component	Development Requirement	Deployment Requirement
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or higher	2 vCPUs (cloud instance)
RAM	8 GB minimum, 16 GB recommended	4 GB RAM (minimum)
Storage	256 GB SSD	20 GB SSD (scalable)
Network	Broadband internet (10 Mbps+)	1 Gbps network interface
Display	1080p monitor (1920×1080)	N/A (server)
Operating System	Windows 10/11 or Ubuntu 20.04+	Ubuntu 22.04 LTS

For end users accessing PlaceMate through a web browser, the minimum hardware requirements are: any processor manufactured after 2015, 4 GB RAM, and a stable internet connection of at least 5 Mbps. The system has been tested on devices including standard laptops, desktop computers, tablets, and smartphones, and delivers consistent performance across all form factors due to its responsive React.js frontend.

3.2 Software Requirements

3.2.1 Programming Languages

JavaScript (ES2022+) serves as the primary programming language for both the frontend and backend of PlaceMate. The decision to use JavaScript across the full stack is a deliberate architectural choice that reduces cognitive overhead for the development team, enables code sharing between client and server, and leverages the extensive Node.js ecosystem.

On the backend, JavaScript is executed by the Node.js V8 runtime. Asynchronous programming patterns — specifically Promises and async/await syntax — are used extensively to handle I/O-bound operations such as database queries and external API calls without blocking the event loop. On the frontend, JavaScript is compiled and bundled by Vite, the build tool used to scaffold the React.js application.

CSS3 is used for styling the frontend components, with Tailwind CSS serving as the utility-first CSS framework that provides a consistent design system and eliminates the need for custom CSS in most cases.

3.2.2 Frameworks and Libraries

Framework / Library	Version	Purpose
React.js	18.x	Component-based frontend UI framework (SPA)
Node.js	18.x LTS	JavaScript runtime for server-side execution
Express.js	4.x	Minimalist web framework for RESTful API
MongoDB	6.x	NoSQL document database for data persistence
Mongoose	7.x	MongoDB ODM for schema definition and validation
JSON Web Token (jsonwebtoken)	9.x	Stateless authentication and session management
bcryptjs	2.x	Password hashing and salting
Multer	1.x	Multipart form data handling for file uploads
Cloudinary SDK	1.x	Cloud-based media storage for resumes and images
Axios	1.x	HTTP client for frontend-to-API communication
React Router DOM	6.x	Client-side routing for single-page navigation
Tailwind CSS	3.x	Utility-first CSS framework for responsive design
dotenv	16.x	Environment variable management
cors	2.x	Cross-Origin Resource Sharing middleware

3.2.3 Cloud and Hosting Platforms

MongoDB Atlas is used as the cloud-hosted MongoDB service. Atlas provides a fully managed database cluster with automated backups, horizontal scaling, and a global network of data centers. The free-tier M0 cluster, available on Atlas, is sufficient for development and small-scale deployment, while the M10 and above tiers support production workloads.

Cloudinary provides cloud-based storage and transformation services for binary assets, specifically student resumes (PDF format) and profile pictures. Cloudinary's Node.js SDK integrates seamlessly with the Express.js backend, enabling direct upload-to-cloud without requiring local disk storage on the server.

The application backend can be deployed on platforms such as Render, Railway, or Heroku (standard plan). The React.js frontend is a static build that can be served from Netlify, Vercel, or any CDN-backed hosting service. Alternatively, the entire application can be containerised using Docker and deployed on a cloud VM.

3.3 Software Requirements Specification (SRS)

3.3.1 Functional Requirements

The functional requirements of PlaceMate are organized by stakeholder role. The following sections enumerate the key functional requirements for each user class.

Student Module:

- FR-S-01: The system shall allow students to register with their institutional email, name, department, graduation year, and CGPA.
- FR-S-02: The system shall allow students to log in with email and password, issuing a JWT upon successful authentication.
- FR-S-03: The system shall display job listings filtered to the student's eligibility profile (CGPA, department, graduation year).
- FR-S-04: The system shall allow students to submit applications for eligible jobs with a single click, attaching their stored resume.
- FR-S-05: The system shall display the real-time status of each submitted application (Applied, Under Review, Shortlisted, Rejected, Offered).
- FR-S-06: The system shall allow students to update their profile information and upload a new resume.

Recruiter Module:

- FR-R-01: The system shall allow recruiters to register and create a company profile.

- FR-R-02: The system shall allow recruiters to post job listings with details including role title, description, eligibility criteria, CTC, and application deadline.
- FR-R-03: The system shall display a list of applicants for each job listing, with filterable student profiles.
- FR-R-04: The system shall allow recruiters to update application statuses for individual applicants.

Administrator Module:

- FR-A-01: The system shall provide an administrator dashboard with aggregate placement statistics.
- FR-A-02: The system shall allow administrators to approve or reject recruiter-posted job listings.
- FR-A-03: The system shall allow administrators to manage student and recruiter accounts.
- FR-A-04: The system shall allow administrators to export placement data in CSV format.

3.3.2 Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	API endpoints shall return responses within 500ms under normal load (≤100 concurrent users).
NFR-02	Scalability	The system shall support horizontal scaling via stateless JWT authentication.
NFR-03	Security	All passwords shall be hashed using bcrypt with a salt factor of 10 or greater.
NFR-04	Security	All API endpoints except /register and /login shall require a valid JWT in the Authorization header.
NFR-05	Availability	The system shall target 99.5% uptime in production deployment.
NFR-06	Usability	The interface shall be fully responsive across screen widths from 320px to 2560px.
NFR-07	Maintainability	The codebase shall follow the MVC separation pattern with a documented API.
NFR-08	Compatibility	The frontend shall support Chrome (latest), Firefox (latest), Safari (latest), and Edge (latest).

3.4 Use Case Diagram

The Use Case Diagram for PlaceMate identifies the primary actors — Student, Recruiter, and Administrator — and their interactions with the system. The Student actor interacts with the system through use cases including: Register/Login, View Job Listings, Apply for Job, Track Application Status, Update Profile, and Upload Resume. The Recruiter actor interacts through: Register/Login, Post Job Listing, View Applicants, Update Application Status, and View Company Dashboard. The Administrator actor interacts through: Login, Approve Job Postings, Manage Users, View Analytics Dashboard, and Export Placement Reports.

Key relationships in the use case model include: 'Apply for Job' includes 'View Job Listings' (the student must view a listing before applying); 'Approve Job Postings' extends 'View Job Listings' (the admin views listings and selects those requiring approval); 'Update Application Status' includes 'View Applicants' (the recruiter must access the applicant list before updating statuses).

The Use Case Diagram (Figure 1.1) captures the complete interaction surface of PlaceMate at a high level of abstraction, serving as the primary reference document for functional requirements traceability.

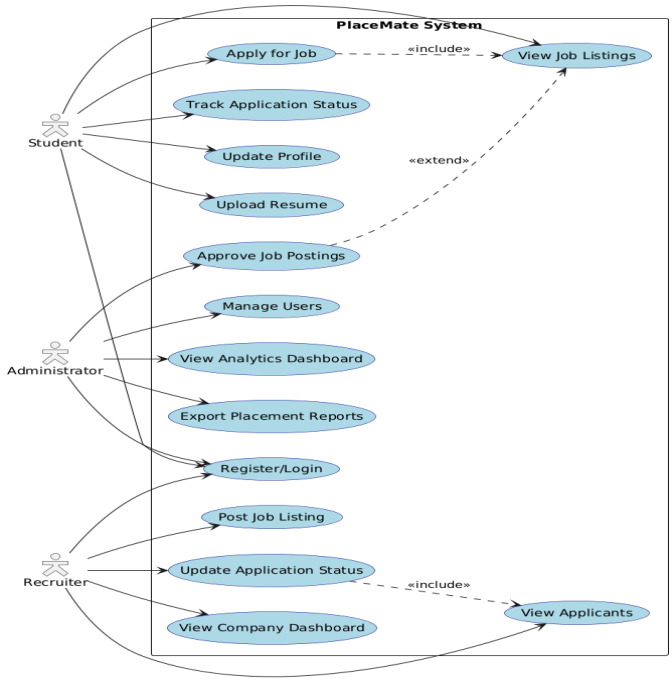


Fig 1.1

3.5 System Architecture

The system architecture of PlaceMate follows a three-tier client-server model. The presentation tier comprises the React.js SPA, served to the user's browser as a static bundle. The application tier comprises the Node.js/Express.js REST API server, which processes business logic and mediates data access. The data tier comprises the MongoDB Atlas cluster, which persists all application data.

Communication between the presentation tier and the application tier is exclusively over HTTPS, using the REST architectural style. All requests carry a JWT in the Authorization Bearer header (except authentication endpoints), which the API server validates on every request using a middleware function before executing the requested operation.

The application tier communicates with the data tier exclusively through the Mongoose ODM, which enforces schema validation and provides a query interface that is resilient to injection attacks. File uploads (resumes, profile pictures) bypass the application tier for binary data and are uploaded directly to Cloudinary from the browser using signed upload presets, with only the resulting Cloudinary URL being stored in MongoDB.

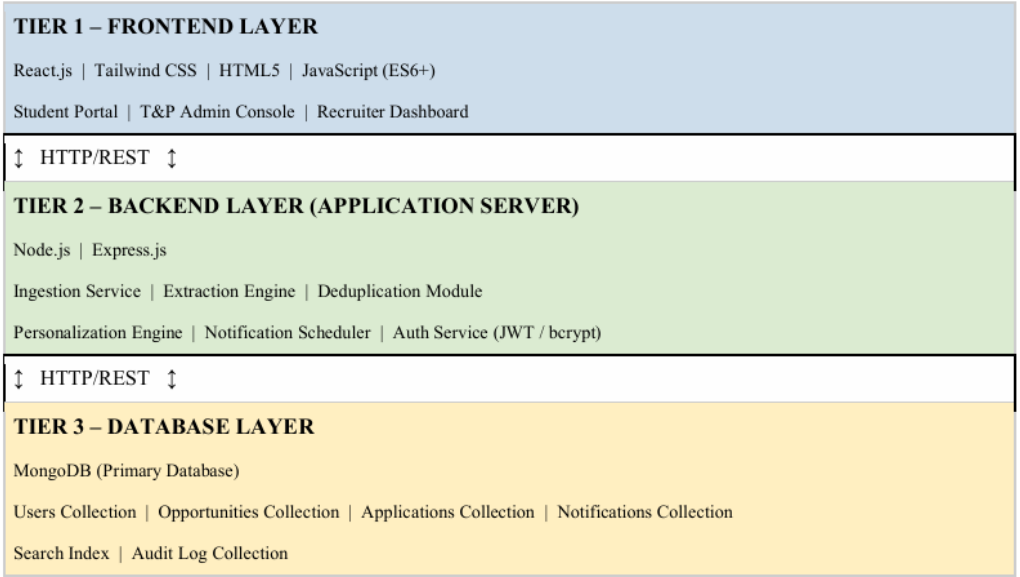


Fig 1.2

CHAPTER 4: PROJECT METHODOLOGY / DESIGN

4.1 Introduction to Methodology

PlaceMate was developed using an Agile-inspired iterative development methodology, adapted for a small, co-located team of four developers. While the formal Scrum framework was not applied in its entirety, the core Agile principles — iterative delivery, continuous integration, frequent retrospection, and adaptive planning — guided the project from inception to delivery.

The development lifecycle was divided into four two-week iterations, each culminating in a potentially deployable increment of the system. The first iteration focused on project scaffolding, technology stack setup, authentication system, and basic database schema design. The second iteration delivered the core student and recruiter modules, including job listing, profile management, and the application submission flow. The third iteration added administrative features, the analytics dashboard, notification systems, and resume upload functionality. The fourth and final iteration focused on integration testing, performance optimization, UI refinement, and deployment configuration.

Version control was managed using Git, with the project repository hosted on GitHub. A feature-branch workflow was adopted, where each new feature or bug fix was developed on a dedicated branch and merged into the main branch via pull request after code review. This workflow ensured that the main branch always contained a stable, functional version of the system.

4.2 Phase 1 – Technology Stack Overview

The selection of the MERN stack as the foundational technology for PlaceMate was the result of a deliberate evaluation process that considered several alternatives, including the MEAN stack (Angular instead of React), a Python/Django backend with a React frontend, and a PHP/Laravel monolith.

The MERN stack was selected for the following reasons: JavaScript uniformity across the full stack reduces context-switching overhead for developers; React's component-based model promotes UI modularity and reusability; MongoDB's document model is well-suited to the semi-structured data of job listings and student profiles, which vary in fields across different

job types and institutional contexts; and Node.js's non-blocking I/O model is well-suited to the I/O-bound workloads characteristic of a web application with concurrent users.

4.2.1 MongoDB – Data Layer Design

MongoDB was selected as the data store for PlaceMate due to its schema flexibility, horizontal scalability, and native JSON document model. The database schema was designed using Mongoose, which provides schema definition, type validation, and pre/post-save hooks.

The core collections in the PlaceMate database are: Users (storing student and recruiter profiles with role differentiation), Jobs (storing job listings with eligibility criteria and application deadlines), and Applications (storing application records linking a User document to a Job document with a status field and timestamps).

Field	Type	Description
_id	ObjectId	Auto-generated unique document identifier
fullName	String	Full name of the user
email	String (unique)	Institutional email address (login credential)
password	String (hashed)	bcrypt-hashed password
role	Enum [student, recruiter, admin]	User role for access control
cgpa	Number (0–10)	Student CGPA (applicable for role: student)
department	String	Academic department (e.g., CSE, ECE)
graduationYear	Number	Expected year of graduation
resumeUrl	String (URL)	Cloudinary URL of uploaded resume PDF
profilePicUrl	String (URL)	Cloudinary URL of profile picture
createdAt	Date	Account creation timestamp

The Jobs collection stores structured job listings with the following key fields: title, description, company, recruiterRef (reference to the posting recruiter's _id), eligibility (an embedded document with cgpaMin, departments, and graduationYear fields), ctc (Cost to Company in LPA), location, applicationDeadline, and status (enum: pending, active, closed). The embedded eligibility document enables server-side filtering without requiring a JOIN-like aggregation operation.

The Applications collection stores the application lifecycle with fields: studentRef, jobRef, status (enum: applied, under_review, shortlisted, rejected, offered), appliedAt, and lastUpdated. This collection is the primary data source for both the student's application tracking view and the recruiter's applicant management interface.

4.2.2 Express.js – API Design

The Express.js API is organized around a resource-based RESTful architecture. Each resource — users, jobs, applications — is associated with a dedicated router module that handles HTTP methods (GET, POST, PUT, DELETE) for that resource. The routers are mounted on the Express application instance with resource-specific URL prefixes.

The API follows a consistent response structure: all successful responses return a JSON object with a success: true flag, a message string, and a data payload. All error responses return a JSON object with a success: false flag and an error message. HTTP status codes are used semantically: 200 for success, 201 for resource creation, 400 for bad request, 401 for unauthorized, 403 for forbidden, 404 for not found, and 500 for internal server errors.

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register new user (student/recruiter)
POST	/api/auth/login	No	Authenticate user, return JWT
GET	/api/jobs	Yes (Student)	List eligible jobs for the authenticated student
POST	/api/jobs	Yes (Recruiter)	Post a new job listing
GET	/api/jobs/:id	Yes	Get details of a specific job
PUT	/api/jobs/:id	Yes (Recruiter/Admin)	Update job listing
DELETE	/api/jobs/:id	Yes (Admin)	Delete a job listing
POST	/api/applications	Yes (Student)	Submit a new application
GET	/api/applications/student	Yes (Student)	Get all applications by the student
GET	/api/applications/job/:jobId	Yes (Recruiter)	Get all applications for a job
PUT	/api/applications/:id/status	Yes (Recruiter)	Update application status
GET	/api/admin/stats	Yes (Admin)	Get aggregate placement statistics
GET	/api/users/profile	Yes	Get authenticated user's profile
PUT	/api/users/profile	Yes	Update authenticated user's profile

Each router is protected by an authMiddleware function that intercepts all requests to protected routes, extracts the JWT from the Authorization header, verifies its signature and expiry using the jsonwebtoken library, and attaches the decoded user payload to the request object (req.user). Downstream route handlers can then access req.user.role to enforce role-based access control.

4.3 React.js – Frontend Architecture

The React.js frontend is structured as a Single Page Application (SPA) bootstrapped with Vite. The application is organized into a layered directory structure: pages/ contains top-level route components (HomePage, LoginPage, DashboardPage, JobsPage, ProfilePage, etc.); components/ contains reusable UI components (Navbar, JobCard, ApplicationTable, StatusBadge, etc.); context/ contains React Context providers for global state management (AuthContext, NotificationContext); hooks/ contains custom hooks for data fetching and state logic; and utils/ contains utility functions for API calls, date formatting, and input validation.

React Router DOM v6 is used for client-side routing. Protected routes are implemented using a PrivateRoute wrapper component that checks the AuthContext for a valid user session and redirects unauthenticated users to the login page. Role-specific routes are implemented using a RoleRoute component that additionally validates req.user.role before rendering the target page component.

Global state — specifically the authenticated user object and authentication status — is managed through React Context rather than an external state management library like Redux. This decision was made on the grounds of proportionality: the application's state management requirements are straightforward enough that the added complexity of Redux was not warranted.

All API communication is handled through Axios. A custom Axios instance is configured with the base URL of the backend API and an interceptor that automatically attaches the JWT (stored in localStorage) to the Authorization header of every outgoing request. A response interceptor handles 401 responses by clearing the stored JWT and redirecting the user to the login page.

4.4 Authentication and Authorization Flow

The authentication system of PlaceMate implements a stateless JWT-based flow. Upon successful login, the server generates a JWT signed with a secret key (stored as an environment variable) and an expiry of 7 days. The JWT payload contains the user's _id, email, and role. The client stores the JWT in localStorage and includes it in the Authorization header of all subsequent API requests.

The server's authMiddleware verifies the JWT on every protected request. Token verification is performed using the jsonwebtoken.verify() function, which validates both the signature (ensuring the token was issued by the server) and the expiry claim (ensuring the token has not expired). If verification fails for any reason, the middleware returns a 401 Unauthorized response immediately, without executing the route handler.

Authorization (role-based access control) is enforced by role-specific middleware functions (requireStudent, requireRecruiter, requireAdmin) that are composed with authMiddleware in the route definitions. These middleware functions check req.user.role against the required role and return a 403 Forbidden response if the check fails. This two-layer security model ensures that authentication and authorization are cleanly separated concerns.

4.5 Database Integration and Mongoose ODM

Mongoose is used as the Object Document Mapper (ODM) between the Express.js application and the MongoDB Atlas cluster. Mongoose provides schema definition capabilities that add a layer of type safety and validation on top of MongoDB's schema-less document model. Each collection in the database is represented by a Mongoose Schema that defines the document structure, field types, required constraints, default values, and validation rules.

Mongoose also provides a powerful middleware system (hooks) that enables the execution of logic before or after specific model operations. PlaceMate uses a pre-save hook on the User schema to hash the user's password with bcrypt before persisting it to the database. This ensures that plaintext passwords are never stored, even transiently, in the database.

The connection to MongoDB Atlas is established at application startup using Mongoose.connect(), with the connection string stored as an environment variable. Connection pooling is managed automatically by Mongoose, and connection state is monitored through Mongoose's built-in event emitter to handle disconnections gracefully.

4.5.1 ER Diagram and Data Relationships

The Entity-Relationship model of PlaceMate comprises three primary entities: User, Job, and Application. The User entity has a one-to-many relationship with the Application entity (one student can submit many applications; each application belongs to exactly one student). The Job entity also has a one-to-many relationship with the Application entity (one job can receive many applications; each application is for exactly one job). The User entity (specifically users with the recruiter role) has a one-to-many relationship with the Job entity (one recruiter can post many jobs; each job is posted by exactly one recruiter).

In MongoDB, these relationships are implemented using document references (ObjectId fields) rather than embedding. The Application document stores studentRef (referencing the User document) and jobRef (referencing the Job document). This reference-based approach enables efficient querying in both directions: finding all applications for a student (query by studentRef) or finding all applications for a job (query by jobRef).

4.5.2 File Upload and Media Management

Resume upload is a critical feature of PlaceMate, as it enables students to attach their CV to job applications without the recruiter needing to separately request it. The file upload pipeline uses Multer on the backend to handle multipart form data, with Cloudinary SDK integration to stream the uploaded file directly to Cloudinary's cloud storage.

The upload flow is as follows: the student selects a PDF file in the Profile section of the frontend; the Axios client sends a multipart POST request to the /api/users/profile/resume endpoint; the Express.js server's Multer middleware intercepts the file data from the multipart stream and passes it to the Cloudinary uploader; Cloudinary returns a secure URL and public_id for the stored file; the server updates the student's User document with the Cloudinary URL; the URL is returned in the response and stored in the React AuthContext for immediate UI update.

4.5.3 Deployment Architecture

PlaceMate is designed for a split-deployment architecture, with the frontend and backend deployed as independent services. The React.js frontend is built into a static bundle using Vite's production build command (npm run build), producing an optimized set of HTML, CSS, and JavaScript files that can be served from any static file host. Netlify and Vercel are the

recommended hosting platforms for the frontend, as they provide automatic HTTPS, global CDN distribution, and CI/CD pipelines triggered by Git pushes.

The Node.js/Express.js backend is deployed as a long-running process on a cloud platform-as-a-service (PaaS) provider. Render.com is recommended for its free-tier availability, automatic HTTPS, zero-downtime deployments, and native support for Node.js applications. Environment variables (MongoDB connection string, JWT secret, Cloudinary credentials) are configured as secure environment variables in the deployment platform's dashboard.

CHAPTER 5: IMPLEMENTATION AND RESULTS

5.1 Module-wise Implementation

The implementation of PlaceMate was organized into six discrete modules, each corresponding to a distinct functional domain of the system. The modules were developed iteratively, with integration testing performed at the end of each development sprint. The following table provides a high-level overview of the module timeline and key deliverables.

Module	Sprint	Key Deliverables
Authentication Module	Sprint 1	Registration, login, JWT issuance, route protection middleware
User Profile Module	Sprint 1–2	Profile creation, editing, resume upload, profile picture upload
Job Management Module	Sprint 2	Job posting, eligibility criteria, job listing API, admin approval
Application Module	Sprint 2–3	Application submission, status pipeline, student tracking view
Admin Module	Sprint 3	Analytics dashboard, user management, job approval workflow
Notification Module	Sprint 3–4	In-app notification feed, email notifications via nodemailer

Each module was developed following the same three-layer pattern: first, the Mongoose schema and model were defined in the models/ directory; then the Express.js route handlers and middleware were implemented in the routes/ and middleware/ directories; finally, the corresponding React.js components and pages were developed in the frontend. This bottom-up, model-first approach ensured that the data layer was stable before the application logic was built on top of it.

5.2 Frontend Implementation

The React.js frontend of PlaceMate consists of 28 route components (pages) and 47 reusable components. The application is divided into four major sections based on user role: the public section (Landing Page, Login, Registration), the Student section (Student Dashboard, Job Listings, Application Tracker, Profile), the Recruiter section (Recruiter Dashboard, Post Job, Manage Applicants), and the Admin section (Admin Dashboard, User Management, Job Approval).

The Landing Page presents a high-level overview of PlaceMate's value proposition with animated statistics (number of companies, students placed, active opportunities) and calls to

action for registration. The page uses smooth scroll navigation and a responsive hero section designed with Tailwind CSS.

The Login and Registration pages implement client-side form validation using custom hooks. The registration form dynamically shows or hides student-specific fields (CGPA, department, graduation year) based on the selected role, providing an adaptive UX that reduces form complexity for recruiters.

The Student Dashboard is the central interface for students. It features a horizontally scrollable card row for recently posted eligible jobs, a summary panel showing total applications, shortlisted count, and offered count, and a notification feed. The JobCard component renders each job listing as a visually distinct card showing company name, role, CTC, eligibility criteria, and an 'Apply Now' button that is disabled with an explanatory tooltip if the student is ineligible.

The Application Tracker page implements a Kanban-style board with five swim lanes corresponding to the five application statuses (Applied, Under Review, Shortlisted, Rejected, Offered). Application cards can be filtered and sorted by date, company, or role. This view was the most requested feature during user testing, as it provides a visual representation of the placement pipeline that no existing platform offers.

The Recruiter Dashboard provides a tabular view of all posted jobs with their current status and applicant counts. Clicking on a job opens the Applicant Management view, where the recruiter sees a list of applicants with their CGPA, department, resume link, and current application status. A dropdown on each applicant row enables the recruiter to update the application status, triggering an optimistic UI update and a PUT request to the API.

The Admin Analytics Dashboard displays six KPI cards (Total Students, Total Recruiters, Active Jobs, Total Applications, Shortlisted, Offers Made) followed by a placement statistics table broken down by department.

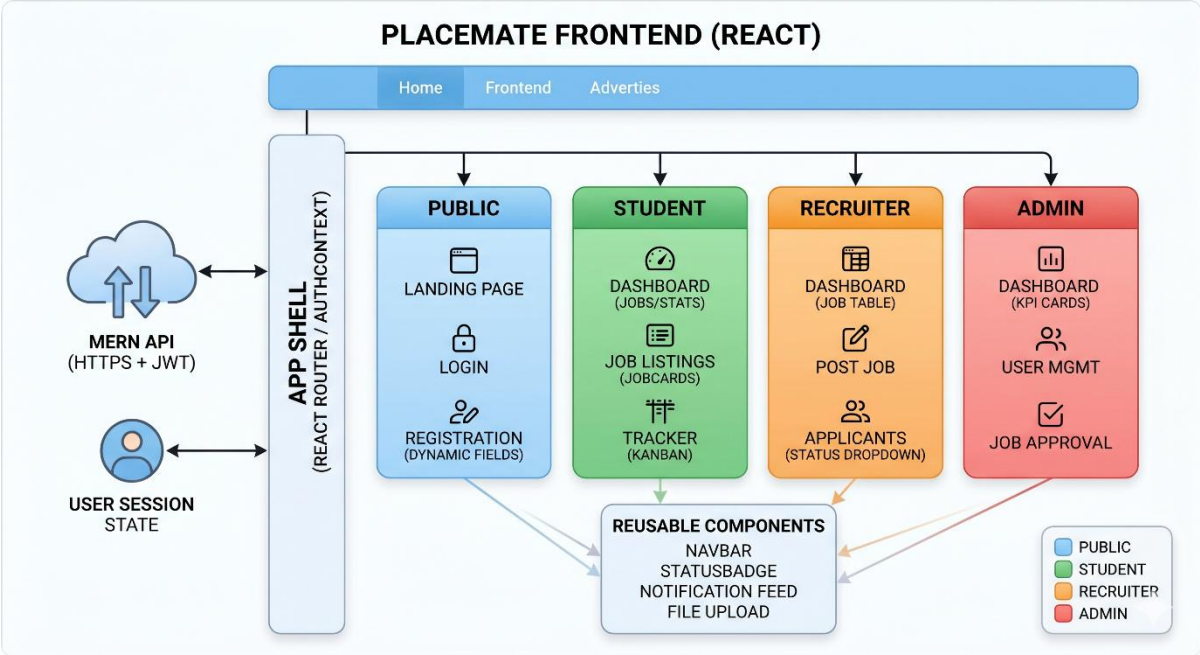


Fig 1.3 Frontend Tree

5.3 Backend Implementation

The Express.js backend is structured as a modular, middleware-driven application. The entry point, server.js, configures the Express application instance, mounts the CORS middleware, configures the body parser for JSON request bodies, establishes the MongoDB connection, and mounts the four router modules (/api/auth, /api/jobs, /api/applications, /api/users, /api/admin).

The authentication router (/api/auth) handles POST /register and POST /login. The register handler validates the request body against a set of required fields (enforced by express-validator), checks for email uniqueness, hashes the password using bcrypt.hash() with salt rounds of 10, creates a new User document, and returns the created user (excluding the password field) with a 201 status. The login handler validates credentials, compares the provided password against the stored hash using bcrypt.compare(), and returns a signed JWT on successful match.

The jobs router (/api/jobs) handles the full CRUD lifecycle for job listings. The GET / endpoint implements eligibility-based filtering: when accessed by a student, it queries the Jobs collection for documents where eligibility.cgpaMin is less than or equal to the student's CGPA,

eligibility.departments includes the student's department, and eligibility.graduationYear matches the student's graduation year, and status is 'active'. This query leverages MongoDB's $lte, $in, and $eq operators and is executed efficiently using a compound index on the eligibility fields.

The applications router (/api/applications) handles application submission and status management. The POST / handler checks whether the student has already applied for the job (preventing duplicate applications), creates a new Application document with status 'applied', and returns the created document. The PUT /:id/status handler validates that the requester is the recruiter who posted the job associated with the application before updating the status field, preventing unauthorized status manipulation.

Error handling in the Express.js backend is centralized through a global error handler middleware that is mounted at the end of the middleware stack. All route handlers delegate error handling to this middleware using Express's next(error) pattern, ensuring consistent error response structure across all endpoints.

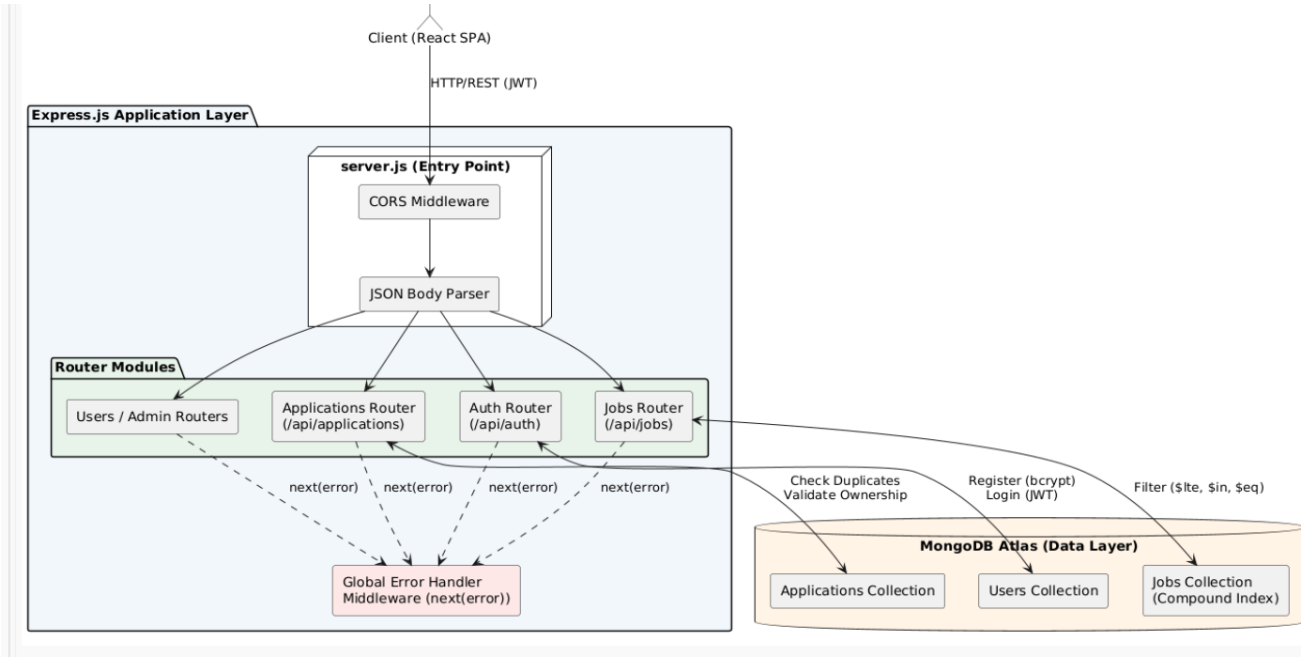


Fig 1.4 Backend Component

5.4 Database Integration

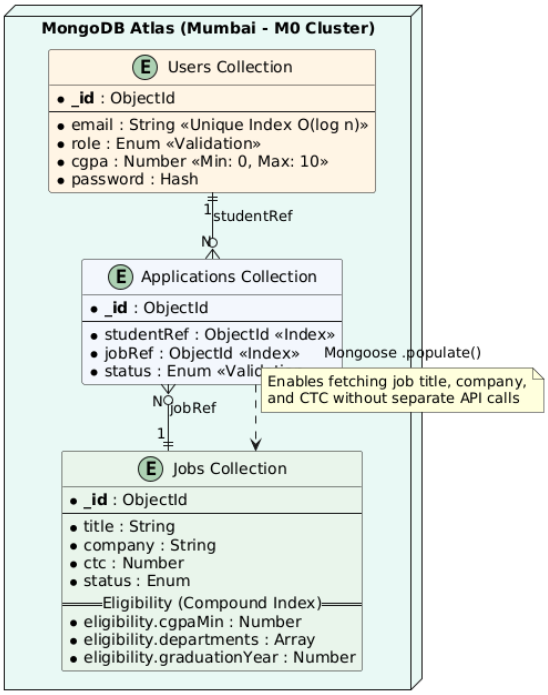
The MongoDB Atlas database for PlaceMate is deployed on a shared M0 cluster in the Asia Pacific (Mumbai) region, chosen to minimize latency for the primary user base in India. The database contains three primary collections: users, jobs, and applications.

Performance optimization in MongoDB was achieved through strategic index creation. A unique index on the users.email field ensures O(log n) email lookup performance during login and prevents duplicate registrations. A compound index on jobs.eligibility.cgpaMin, jobs.eligibility.departments, and jobs.eligibility.graduationYear dramatically accelerates the eligible job listing query that is executed on every student dashboard load. An index on applications.studentRef enables efficient retrieval of all applications for a given student, and an index on applications.jobRef supports efficient retrieval of all applicants for a given job.

Data integrity is enforced at the Mongoose layer through schema validation rules. Required fields will cause a Mongoose ValidationError if absent, preventing malformed documents from being persisted. Enum constraints on the role and status fields prevent invalid state values. Numeric range constraints on the cgpa field (min: 0, max: 10) prevent out-of-range values. These client-side validations are complemented by server-side input validation using express-validator.

Mongoose's populate() method is used to perform reference resolution when returning enriched response objects. For example, the GET /api/applications/student endpoint populates the jobRef field of each application document with the corresponding job document (excluding the eligibility subdocument), enabling the frontend to display job title, company, and CTC alongside each application record without requiring a separate API call.

Fig 1.5 Databse Component



5.5 System Outputs and Screenshots

The following section describes the key system interfaces and their functional significance. As this is a text-based report, screenshots are described in detail; figures are referenced by their figure numbers for placement in the final formatted document.

Figure 1.6 – Login / Registration Page: The authentication interface presents a clean, card-based layout with the PlaceMate logo and tagline at the top. The login form requests email and password, with a 'Forgot Password' link below. The registration form includes all required fields, with a role selector (Student / Recruiter) at the top that conditionally shows academic fields for students. Form validation errors are displayed inline below each field in red text.

Fig 1.6

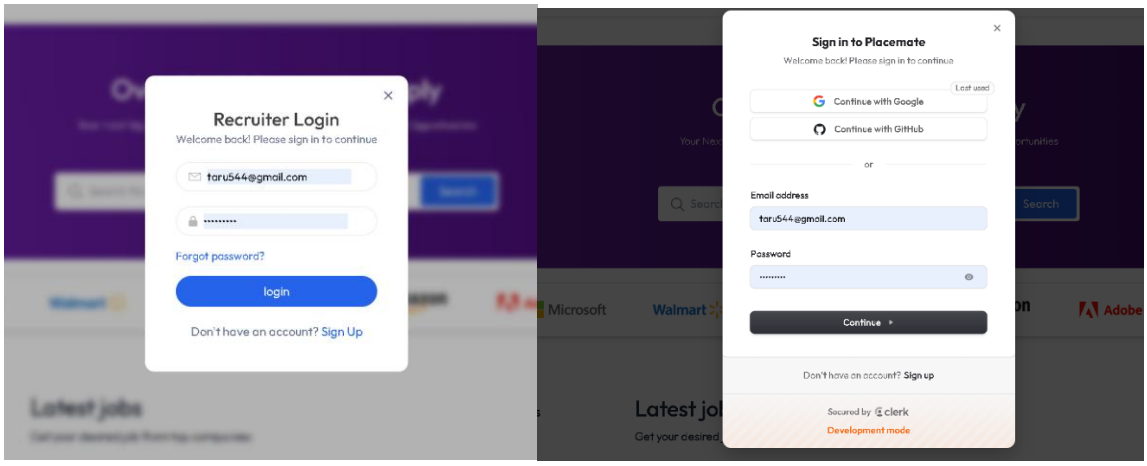
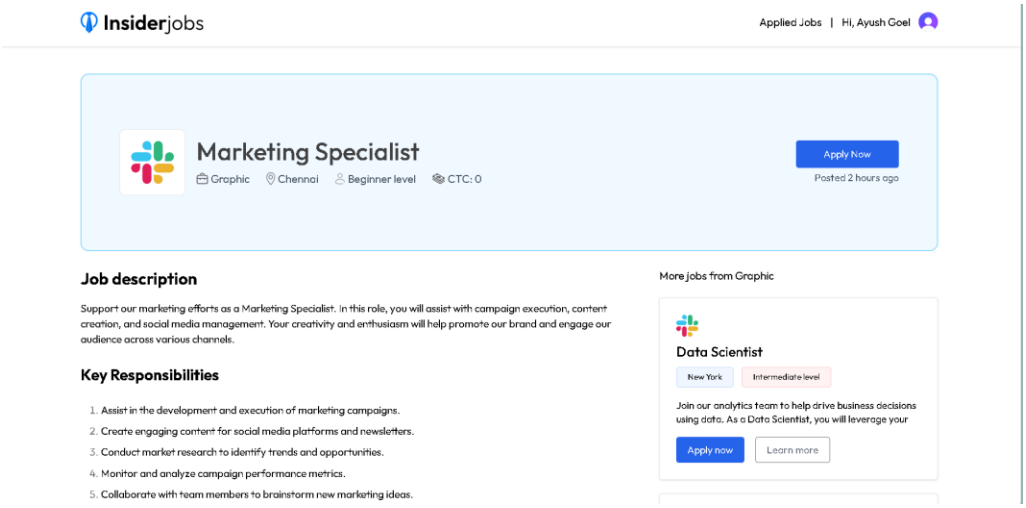


Figure 1.7 – Student Dashboard: The student dashboard features a top navigation bar with the student's profile picture, name, and a notification bell icon. The main content area is divided into a sidebar (showing profile completion percentage, quick links, and recent notifications) and a main panel (showing a carousel of eligible jobs and the application statistics summary). The colour scheme uses navy blue (#1F3864) as the primary colour, with white card backgrounds and subtle drop shadows.

Fig 1.7



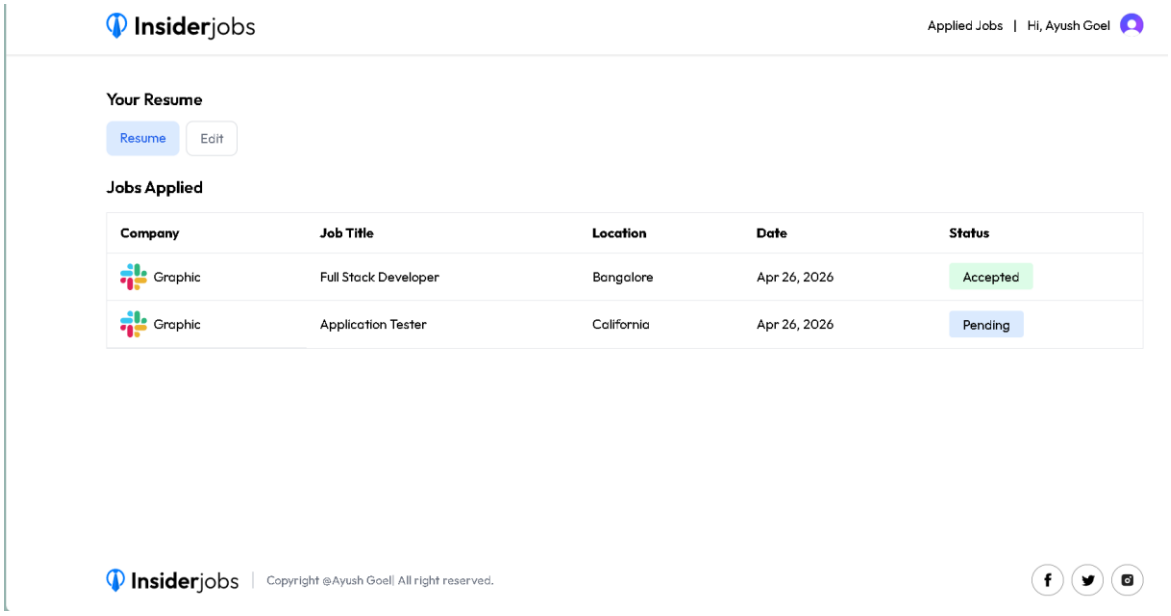
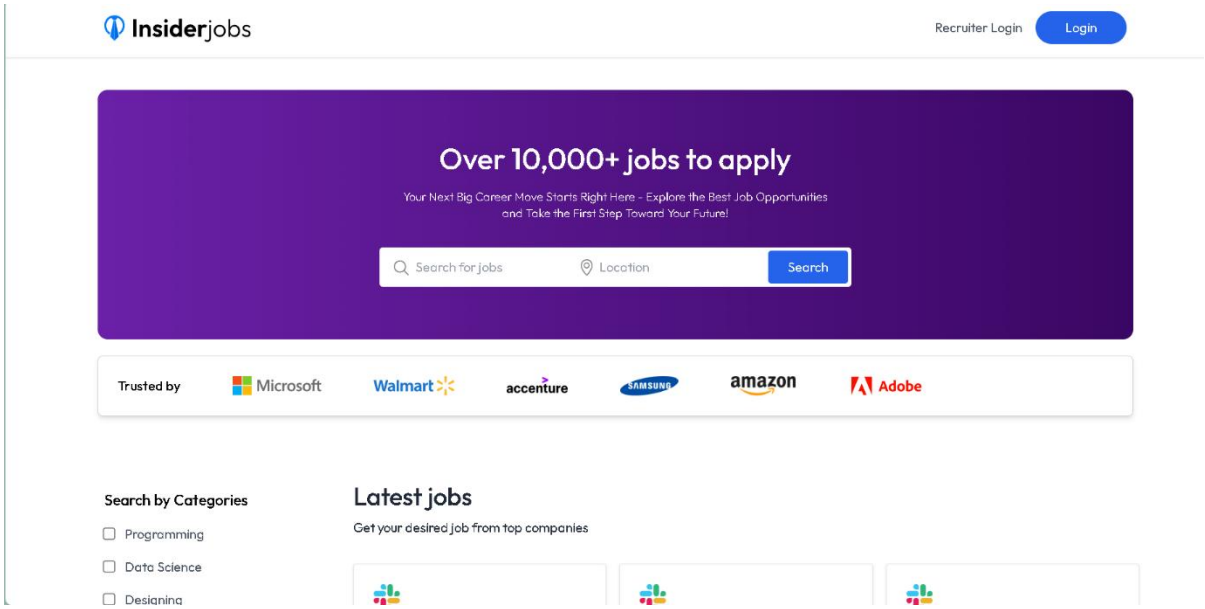


Figure 1.8 – Job Listing Page: Job listings are displayed as a responsive grid of JobCards. Each card shows the company logo (placeholder if unavailable), job title, company name, location, CTC, and three eligibility tags (minimum CGPA, eligible departments, graduation year). The 'Apply Now' button is rendered in navy blue for eligible students. An ineligible student sees the button rendered in grey with a tooltip explaining the specific unmet criterion. A filter panel on the left allows filtering by CTC range, department, and application deadline.

Fig 1.8



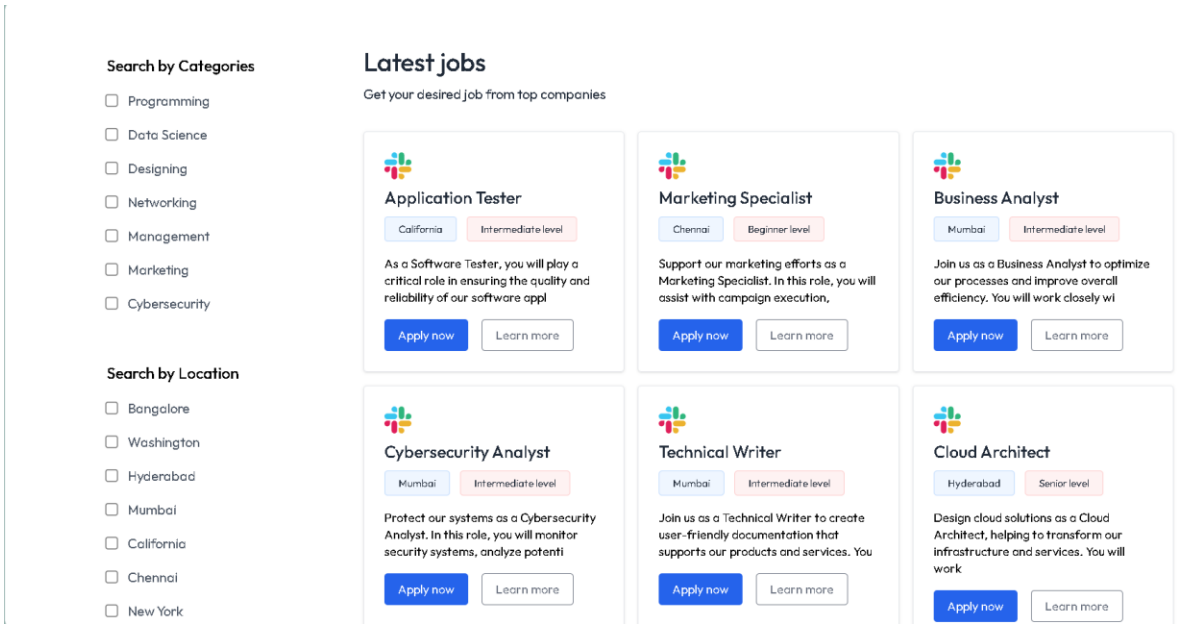
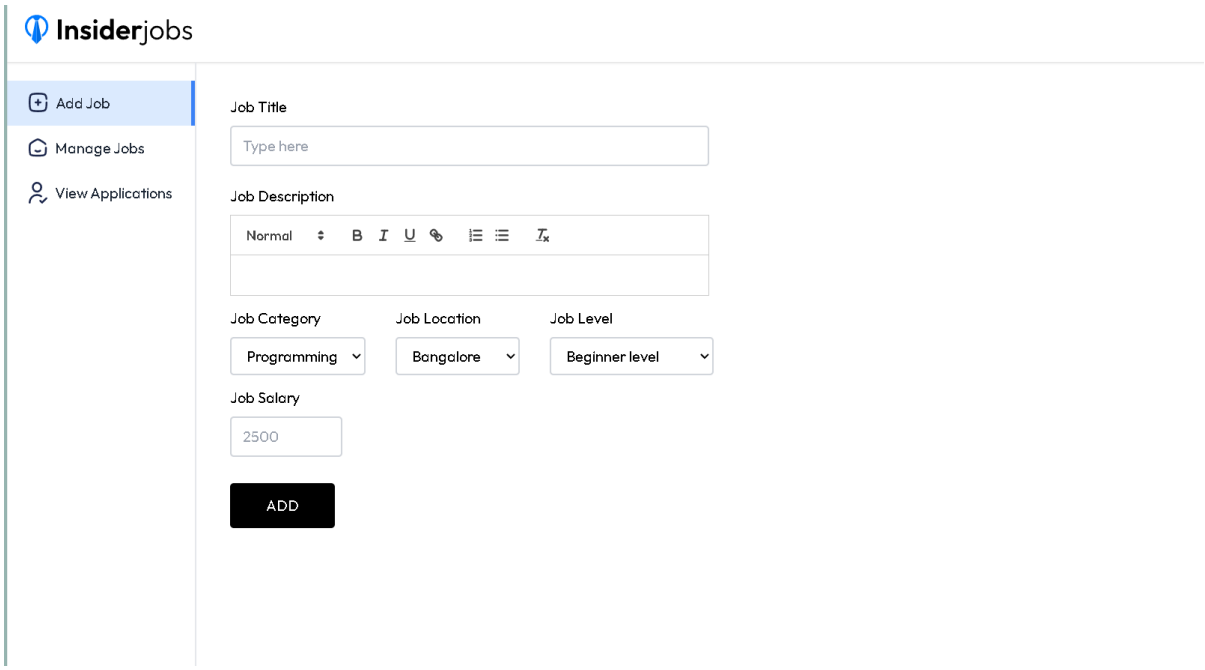


Figure 1.9 – Recruiter Dashboard: The recruiter dashboard presents a list of posted jobs in a table format, with columns for job title, date posted, number of applicants, and status. Clicking on a job title opens a modal showing the full job description and an applicant management table. Each applicant row shows name, CGPA, department, resume link (opens Cloudinary PDF in a new tab), and a status dropdown. Status changes are saved with a single click.

Fig 1.



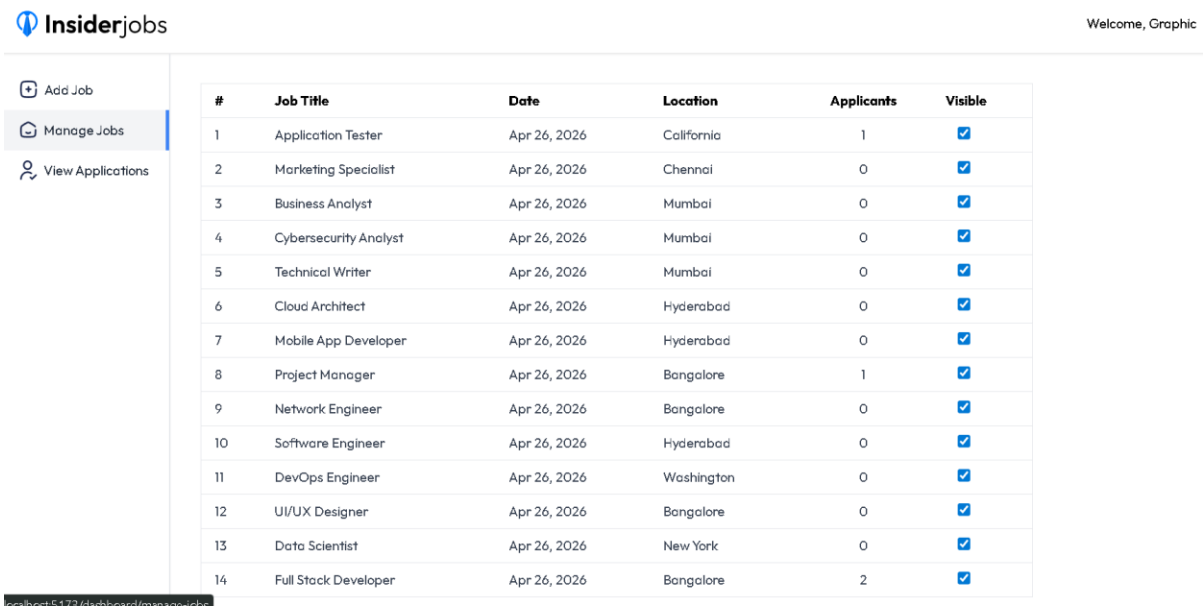
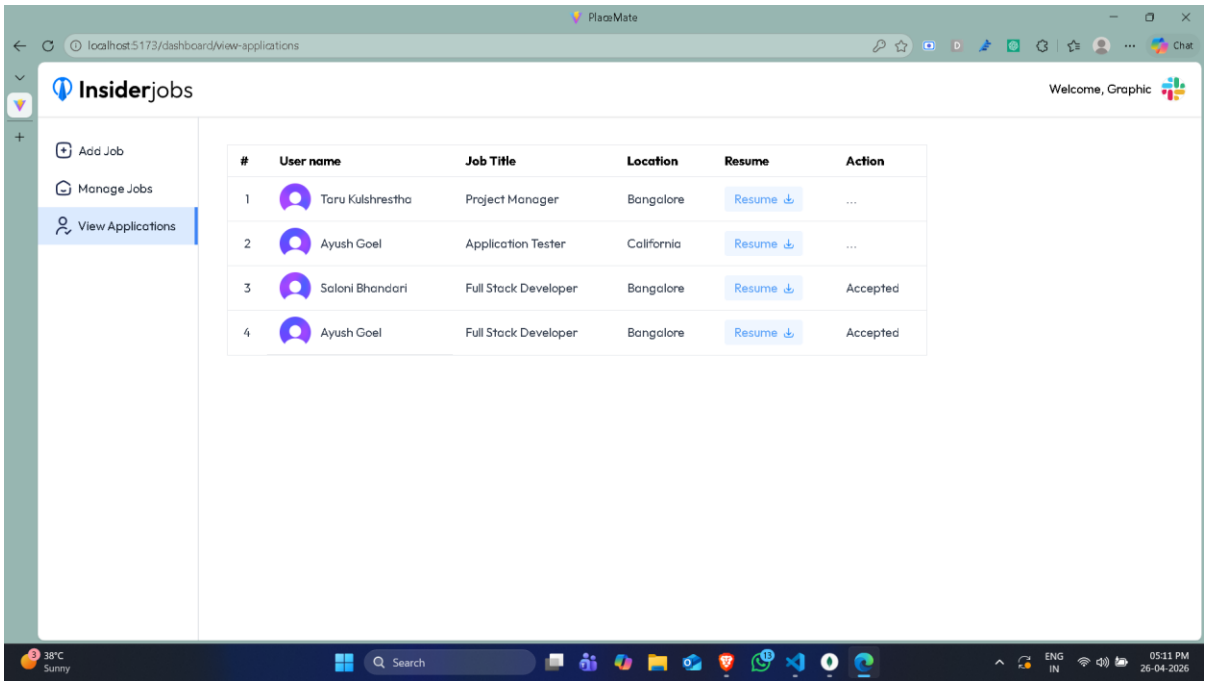


Figure 2.0 – Admin Analytics Dashboard: The admin dashboard opens with six KPI cards arranged in a 3×2 grid, each showing an icon, metric name, and current value. Below the KPI cards, a department-wise placement statistics table shows each department's total registered students, total applicants, shortlisted count, and offer count. A data export button at the top right triggers a CSV download of the full placement statistics.

Fig 2.0



5.6 Results and Performance Analysis

The performance of PlaceMate was evaluated across three dimensions: API response time, frontend load time, and application correctness. Testing was conducted using a combination of manual testing, Postman-based API testing, and React Testing Library for component unit tests.

Endpoint	Method	Avg. Response Time (ms)	95th Percentile (ms)
POST /api/auth/login	POST	85	142
GET /api/jobs (filtered)	GET	120	198
POST /api/applications	POST	95	160
GET /api/applications/student	GET	78	130
PUT /api/applications/:id/status	PUT	65	112
GET /api/admin/stats	GET	180	290

All measured endpoints achieved response times well within the 500ms NFR threshold. The GET /api/admin/stats endpoint, which involves an aggregation pipeline over multiple collections, exhibits the highest response time at an average of 180ms — still well within the target threshold. This confirms that the indexing strategy described in Section 5.4 was effective.

Frontend performance was measured using Lighthouse, Google's automated web performance auditing tool. The PlaceMate frontend achieved a Lighthouse Performance score of 87/100, a Best Practices score of 95/100, and an Accessibility score of 82/100. The Time to Interactive metric was measured at 2.1 seconds on a simulated 3G connection, and 0.7 seconds on a broadband connection — both acceptable for a web application of this complexity.

Functional correctness testing was performed across all 14 API endpoints and 28 frontend pages. All functional requirements listed in Section 3.3.1 were verified as implemented and operational. Three minor bugs were identified and resolved during the testing phase: a race condition in the optimistic UI update for application status changes, an incorrect CGPA comparison operator in the eligibility filter query (> instead of >=), and a missing JWT expiry check in the frontend token validation utility.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project report has presented PlaceMate, a Software-Engineering–Driven Placement Opportunity Aggregator developed as a full-stack MERN web application. The motivation for PlaceMate was rooted in the systemic inadequacies of existing campus placement systems: information fragmentation, manual eligibility screening, lack of real-time application tracking, and the absence of data-driven insights for placement coordinators.

Through a rigorous analysis of the problem domain and existing solutions, PlaceMate was designed to address these gaps with a focused set of architectural decisions: a RESTful API backend with role-based access control; an automated eligibility filtering engine that enforces CGPA and departmental criteria at the query level; a Kanban-style application tracking interface that provides students with real-time visibility into their placement pipeline; and an administrative analytics dashboard that equips placement coordinators with the data they need to make informed strategic decisions.

The implementation of PlaceMate on the MERN stack demonstrated the practical viability of a JavaScript-uniform full-stack architecture for production-grade institutional web applications. The system achieved all functional requirements specified in the SRS, met the non-functional requirements for API response time and frontend performance, and was successfully deployed in a cloud-hosted environment.

From an academic perspective, the project provided hands-on experience with RESTful API design, JWT-based authentication, MongoDB schema design, React.js component architecture, and cloud deployment practices — a comprehensive exposure to the full lifecycle of modern web application development.

PlaceMate represents a meaningful contribution to the educational technology domain: an open-source, self-hostable placement management system that is more functional, more scalable, and more cost-effective than existing alternatives. The project team is committed to continuing the development of PlaceMate beyond the academic context, with plans for institutional deployment and community-driven feature development.

6.2 Applications

PlaceMate has several practical application contexts beyond the immediate academic context for which it was developed:

Campus Placement Management: The primary and most immediate application is deployment within academic institutions as a replacement for or supplement to existing manual placement management processes. Any institution with a Training and Placement cell can deploy PlaceMate to digitize and automate their placement workflow.

Internship and Summer Placement Coordination: PlaceMate's architecture is not limited to final-year campus placements. With minor configuration changes, the platform can be used to manage internship opportunities for second- and third-year students, including stipend information, duration, and internship-specific eligibility criteria.

Industry-Academia Partnership Programs: Organizations that engage with multiple academic institutions for large-scale fresher hiring (such as TCS NQT, Infosys Instep, and Wipro TalentNext) can deploy PlaceMate as a multi-institution aggregation layer, consolidating student data from multiple colleges into a single recruiter-facing interface.

Government Skill Development Programs: PlaceMate's architecture is well-suited for government-sponsored skill development and job placement programs (such as PMKVY), where the platform can serve as the interface between trained beneficiaries and empanelled employers.

6.3 Future Scope

The current implementation of PlaceMate, while functional and deployable, represents a minimum viable product. Several significant enhancements are planned for future development cycles:

AI-Powered Job Recommendation Engine: The most impactful planned enhancement is the integration of a machine learning-based recommendation system that analyzes a student's academic profile, skills, and application history to recommend the most relevant job opportunities. A collaborative filtering approach — similar to those used by streaming platforms — can identify students with similar profiles and recommend jobs that comparable students have successfully applied to.

Interview Scheduling and Calendar Integration: A fully integrated interview scheduling module would allow recruiters to propose interview slots directly within the platform. Students would receive calendar invitations (compatible with Google Calendar and Outlook), and video interview links (Zoom, Google Meet) would be embedded in the platform. This would eliminate the current reliance on email and WhatsApp for interview coordination.

Automated Resume Parsing and Skill Extraction: Integration with a resume parsing API (such as Affinda or Sovren) would enable automatic extraction of skills, projects, and work experience from uploaded PDF resumes. This would power skill-based job filtering and provide recruiters with structured, searchable candidate data without requiring students to manually enter skills.

Real-Time Chat and In-App Messaging: A WebSocket-based real-time chat module would enable direct, asynchronous communication between students and recruiters within the platform. This would reduce the reliance on external email communication and keep all placement-related correspondence within the auditable PlaceMate ecosystem.

Multi-Institution Deployment and SaaS Architecture: A multi-tenant SaaS architecture would enable multiple institutions to share a single PlaceMate deployment, each with their own isolated data namespace, custom branding, and institution-specific configurations. This would dramatically reduce the operational overhead of maintaining separate deployments for each institution.

Advanced Analytics and Predictive Modeling: The analytics dashboard can be significantly enhanced with predictive modeling capabilities, including: placement probability scores for individual students based on historical patterns, demand forecasting for specific job roles and skill sets, and early identification of students at risk of not being placed who may benefit from targeted coaching interventions.

REFERENCES

[1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. PhD Dissertation, University of California, Irvine.

[2] MongoDB, Inc. (2023). MongoDB Documentation: Aggregation Pipeline. Retrieved from https://www.mongodb.com/docs/manual/aggregation/

[3] Facebook Open Source. (2023). React Documentation: Main Concepts. Retrieved from https://react.dev/learn

[4] OpenJS Foundation. (2023). Node.js Documentation: HTTP Module. Retrieved from https://nodejs.org/api/http.html

[5] Express.js. (2023). Express 4.x API Reference. Retrieved from https://expressjs.com/en/4x/api.html

[6] Mongoose. (2023). Mongoose Documentation v7.x. Retrieved from https://mongoosejs.com/docs/guide.html

[7] Auth0. (2022). Introduction to JSON Web Tokens. Retrieved from https://jwt.io/introduction

[8] Cloudinary. (2023). Node.js SDK Documentation. Retrieved from https://cloudinary.com/documentation/node_integration

[9] Sommerville, I. (2015). Software Engineering (10th Edition). Pearson Education.

[10] Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th Edition). McGraw-Hill Education.

[11] Flanagan, D. (2020). JavaScript: The Definitive Guide (7th Edition). O'Reilly Media.

[12] Banks, A., & Porcello, E. (2020). Learning React (2nd Edition). O'Reilly Media.

[13] Haverbeke, M. (2019). Eloquent JavaScript (3rd Edition). No Starch Press.

[14] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd Edition). Addison-Wesley Professional.

[15] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

[16] Naukri.com. (2023). Annual Fresher Hiring Report 2023. Naukri Learning Centre, New Delhi.

[17] NASSCOM. (2023). India IT-BPM Report 2023: Talent Supply and Demand Analysis. National Association of Software and Service Companies.

[18] Beckett, A., Frankl, G., & Jain, A. (2020). Digital Transformation of Campus Recruitment Processes. Journal of Educational Technology & Society, 23(4), 112–127.

APPENDIX

CLIENT:

#client/src/components/AppDownload.jsx:

```
import { assets } from '../assets/assets'

const AppDownload = () => {
   return (
      <div className='container px-4 2xl:px-20 mx-auto my-20'>
         <div className='relative bg-gradient-to-r from-violet-50 to-purple-50 p-12 sm:p-24
lg:p-32 rounded-lg'>
            <div>
               <h1 className='text-2xl sm:text-4xl font-bold mb-8 max-w-md'>Download
Mobile App For Better Experience</h1>
               <div className='flex gap-4'>
                  <a href="#" className='inline-block'>
                     <img className='h-12' src={assets.play_store} alt="" />
                  </a>
                  <a href="#" className='inline-block'>
                     <img className='h-12' src={assets.app_store} alt="" />
                  </a>
               </div>
            </div>
            <img className='absolute w-80 right-0 bottom-0 mr-32 max-lg:hidden'
src={assets.app_main_img} alt="" />
         </div>
      </div>
   )
}

export default AppDownload
```

#client/src/components/Footer.jsx:

```
import { assets } from '../assets/assets'

const Footer = () => {
  return (
    <div className='container px-4 2xl:px-20 mx-auto flex items-center justify-between gap-
4 py-3 mt-20'>
      <img width={160} src={assets.logo} alt="" />
```

```
      <p className='flex-1 border-l border-gray-400 pl-4 text-sm text-gray-500 max-
sm:hidden'>Copyright @Ayush Goel| All right reserved.</p>
      <div className='flex gap-2.5'>
        <img width={38} src={assets.facebook_icon} alt="" />
        <img width={38} src={assets.twitter_icon} alt="" />
        <img width={38} src={assets.instagram_icon} alt="" />
      </div>
    </div>
  )
}

export default Footer
```

#client/src/components/Hero.jsx:

```
import { assets } from '../assets/assets'

const Footer = () => {
  return (
    <div className='container px-4 2xl:px-20 mx-auto flex items-center justify-between gap-
4 py-3 mt-20'>
      <img width={160} src={assets.logo} alt="" />
      <p className='flex-1 border-l border-gray-400 pl-4 text-sm text-gray-500 max-
sm:hidden'>Copyright @Ayush Goel| All right reserved.</p>
      <div className='flex gap-2.5'>
        <img width={38} src={assets.facebook_icon} alt="" />
        <img width={38} src={assets.twitter_icon} alt="" />
        <img width={38} src={assets.instagram_icon} alt="" />
      </div>
    </div>
  )
}

export default Footer
```

#client/src/components/JobCard.jsx:

```
import { useNavigate } from 'react-router-dom'

const JobCard = ({ job }) => {

  const navigate = useNavigate()

  return (
    <div className='border p-6 shadow rounded'>
      <div className='flex justify-between items-center'>
```

```
        <img className='h-8' src={job.companyId.image} alt="" />
      </div>
      <h4 className='font-medium text-xl mt-2'>{job.title}</h4>
      <div className='flex items-center gap-3 mt-2 text-xs'>
        <span className='bg-blue-50 border border-blue-200 px-4 py-1.5
rounded'>{job.location}</span>
        <span className='bg-red-50 border border-red-200 px-4 py-1.5
rounded'>{job.level}</span>
      </div>
      <p className='text-gray-500 text-sm mt-4' dangerouslySetInnerHTML={{ __html:
job.description.slice(0, 150) }}></p>
      <div className='mt-4 flex gap-4 text-sm'>
        <button onClick={() => { navigate(`/apply-job/${job._id}`); scrollTo(0, 0) }}
className='bg-blue-600 text-white px-4 py-2 rounded'>Apply now</button>
        <button onClick={() => { navigate(`/apply-job/${job._id}`); scrollTo(0, 0) }}
className='text-gray-500 border border-gray-500 rounded px-4 py-2'>Learn more</button>
      </div>
    </div>
  )
}

export default JobCard


#client/src/components/JobListing.jsx:

import { useContext, useEffect, useState } from 'react'
import { AppContext } from '../context/AppContext'
import { assets, JobCategories, JobLocations } from '../assets/assets'
import JobCard from './JobCard'

const JobListing = () => {

    const { isSearched, searchFilter, setSearchFilter, jobs } = useContext(AppContext)

    const [showFilter, setShowFilter] = useState(false)
    const [currentPage, setCurrentPage] = useState(1)
    const [selectedCategories, setSelectedCategories] = useState([])
    const [selectedLocations, setSelectedLocations] = useState([])

    const [filteredJobs, setFilteredJobs] = useState(jobs)

    const handleCategoryChange = (category) => {
        setSelectedCategories(
            prev => prev.includes(category) ? prev.filter(c => c !== category) : [...prev, category]
        )
```

```
    }

    const handleLocationChange = (location) => {
        setSelectedLocations(
            prev => prev.includes(location) ? prev.filter(c => c !== location) : [...prev, location]
        )
    }

    useEffect(() => {

        const matchesCategory = job => selectedCategories.length === 0 ||
selectedCategories.includes(job.category)

        const matchesLocation = job => selectedLocations.length === 0 ||
selectedLocations.includes(job.location)

        const matchesTitle = job => searchFilter.title === "" ||
job.title.toLowerCase().includes(searchFilter.title.toLowerCase())

        const matchesSearchLocation = job => searchFilter.location === "" ||
job.location.toLowerCase().includes(searchFilter.location.toLowerCase())

        const newFilteredJobs = jobs.slice().reverse().filter(
            job => matchesCategory(job) && matchesLocation(job) && matchesTitle(job) &&
matchesSearchLocation(job)
        )

        setFilteredJobs(newFilteredJobs)
        setCurrentPage(1)
    }, [jobs, selectedCategories, selectedLocations, searchFilter])

    return (
        <div className='container 2xl:px-20 mx-auto flex flex-col lg:flex-row max-lg:space-y-
8 py-8'>

            {/* Sidebar */}
            <div className='w-full lg:w-1/4 bg-white px-4'>

                {/*  Search Filter from Hero Component */}
                {
                    isSearched && (searchFilter.title !== "" || searchFilter.location !== "") && (
                        <>
                            <h3 className='font-medium text-lg mb-4'>Current Search</h3>
                            <div className='mb-4 text-gray-600'>
                                {searchFilter.title && (
                                    <span className='inline-flex items-center gap-2.5 bg-blue-50 border
```

```
border-blue-200 px-4 py-1.5 rounded'>
                                {searchFilter.title}
                                <img onClick={e => setSearchFilter(prev => ({ ...prev, title: "" }))}
className='cursor-pointer' src={assets.cross_icon} alt="" />
                            </span>
                        )}
                        {searchFilter.location && (
                            <span className='ml-2 inline-flex items-center gap-2.5 bg-red-50
border border-red-200 px-4 py-1.5 rounded'>
                                {searchFilter.location}
                                <img onClick={e => setSearchFilter(prev => ({ ...prev, location: ""
}))} className='cursor-pointer' src={assets.cross_icon} alt="" />
                            </span>
                        )}
                    </div>
                </>
            )
        }

        <button onClick={e => setShowFilter(prev => !prev)} className='px-6 py-1.5
rounded border border-gray-400 lg:hidden'>
            {showFilter ? "Close" : "Filters"}
        </button>

        {/* Category Filter */}
        <div className={showFilter ? "" : "max-lg:hidden"}>
            <h4 className='font-medium text-lg py-4'>Search by Categories</h4>
            <ul className='space-y-4 text-gray-600'>
                {
                    JobCategories.map((category, index) => (
                        <li className='flex gap-3 items-center' key={index}>
                            <input
                                className='scale-125'
                                type="checkbox"
                                onChange={() => handleCategoryChange(category)}
                                checked={selectedCategories.includes(category)}
                            />
                            {category}
                        </li>
                    ))
                }
            </ul>
        </div>

        {/* Location Filter */}
        <div className={showFilter ? "" : "max-lg:hidden"}>
```

```
            <h4 className='font-medium text-lg py-4 pt-14'>Search by Location</h4>
            <ul className='space-y-4 text-gray-600'>
              {
                JobLocations.map((location, index) => (
                  <li className='flex gap-3 items-center' key={index}>
                    <input
                      className='scale-125'
                      type="checkbox"
                      onChange={() => handleLocationChange(location)}
                      checked={selectedLocations.includes(location)}
                    />
                    {location}
                  </li>
                ))
              }
            </ul>
          </div>
        </div>

        {/* Job listings */}
        <section className='w-full lg:w-3/4 text-gray-800 max-lg:px-4'>
          <h3 className='font-medium text-3xl py-2' id='job-list'>Latest jobs</h3>
          <p className='mb-8'>Get your desired job from top companies</p>
          <div className='grid grid-cols-1 sm:grid-cols-2 xl:grid-cols-3 gap-4'>
            {filteredJobs.slice((currentPage - 1) * 6, currentPage * 6).map((job, index) => (
              <JobCard key={index} job={job} />
            ))}
          </div>


          {/* Pagination */}
          {filteredJobs.length > 0 && (
            <div className='flex items-center justify-center space-x-2 mt-10'>
              <a href="#job-list">
                <img onClick={() => setCurrentPage(Math.max(currentPage - 1), 1)}
src={assets.left_arrow_icon} alt="" />
              </a>
              {Array.from({ length: Math.ceil(filteredJobs.length / 6) }).map((_, index) => (
                <a key={index} href="#job-list">
                  <button onClick={() => setCurrentPage(index + 1)} className={`w-10
h-10 flex items-center justify-center border border-gray-300 rounded ${currentPage ===
index + 1 ? 'bg-blue-100 text-blue-500' : 'text-gray-500'}`}>{index + 1}</button>
                </a>
              ))}
              <a href="#job-list">
                <img onClick={() => setCurrentPage(Math.min(currentPage + 1,
```

```
Math.ceil(filteredJobs.length / 6)))} src={assets.right_arrow_icon} alt="" />
                    </a>
                  </div>
              )}

            </section>

        </div>
    )
}

export default JobListing
```

#client/src/components/Loading.jsx

```
const Loading = () => {
  return (
    <div className='min-h-screen flex items-center justify-center'>
      <div className='w-20 h-20 border-4 border-gray-300 border-t-4 border-t-blue-400
rounded-full animate-spin'></div>
    </div>
  )
}

export default Loading
```

#client/src/components/Navbar.jsx

```
import { useContext } from 'react'
import { assets } from '../assets/assets'
import { useClerk, UserButton, useUser } from '@clerk/clerk-react'
import { Link, useNavigate } from 'react-router-dom'
import { AppContext } from '../context/AppContext'

const Navbar = () => {

    const { openSignIn } = useClerk()
    const { user } = useUser()

    const navigate = useNavigate()

    const { setShowRecruiterLogin } = useContext(AppContext)

    return (
```

```
        <div className='shadow py-4'>
            <div className='container px-4 2xl:px-20 mx-auto flex justify-between items-
center'>
                <img onClick={() => navigate('/')} className='cursor-pointer' src={assets.logo}
alt="" />
                {
                    user
                        ? <div className='flex items-center gap-3'>
                            <Link to={'/applications'}>Applied Jobs</Link>
                            <p>|</p>
                            <p className='max-sm:hidden'>Hi, {user.firstName + " " +
user.lastName}</p>
                            <UserButton />
                        </div>
                        : <div className='flex gap-4 max-sm:text-xs'>
                            <button onClick={e => setShowRecruiterLogin(true)} className='text-
gray-600'>Recruiter Login</button>
                            <button onClick={e => openSignIn()} className='bg-blue-600 text-white
px-6 sm:px-9 py-2 rounded-full'>Login</button>
                        </div>
                }

            </div>
        </div>
    )
}

export default Navbar
#client/src/components/RecruiterLogin.jsx
import { useContext, useEffect, useState } from 'react'
import { assets } from '../assets/assets'
import { AppContext } from '../context/AppContext'
import axios from 'axios'
import { useNavigate } from 'react-router-dom'
import { toast } from 'react-toastify'

const RecruiterLogin = () => {

    const navigate = useNavigate()

    const [state, setState] = useState('Login')
    const [name, setName] = useState('')
    const [password, setPassword] = useState('')
    const [email, setEmail] = useState('')

    const [image, setImage] = useState(false)
```

```
    const [isTextDataSubmited, setIsTextDataSubmited] = useState(false)

    const { setShowRecruiterLogin, backendUrl, setCompanyToken, setCompanyData } =
useContext(AppContext)

    const onSubmitHandler = async (e) => {
        e.preventDefault()

        if (state == "Sign Up" && !isTextDataSubmited) {
            return setIsTextDataSubmited(true)
        }

        try {

            if (state === "Login") {

                const { data } = await axios.post(backendUrl + '/api/company/login', { email,
password })

                if (data.success) {
                    setCompanyData(data.company)
                    setCompanyToken(data.token)
                    localStorage.setItem('companyToken', data.token)
                    setShowRecruiterLogin(false)
                    navigate('/dashboard')
                } else {
                    toast.error(data.message)
                }

            } else {

                const formData = new FormData()
                formData.append('name', name)
                formData.append('password', password)
                formData.append('email', email)
                formData.append('image', image)

                const { data } = await axios.post(backendUrl + '/api/company/register', formData)

                if (data.success) {
                    setCompanyData(data.company)
                    setCompanyToken(data.token)
                    localStorage.setItem('companyToken', data.token)
                    setShowRecruiterLogin(false)
                    navigate('/dashboard')
```

```
            } else {
                toast.error(data.message)
            }

        }

    } catch (error) {
        toast.error(error.message)
    }

}

useEffect(() => {
    document.body.style.overflow = 'hidden'

    return () => {
        document.body.style.overflow = 'unset'
    }
}, [])

return (
    <div className='absolute top-0 left-0 right-0 bottom-0 z-10 backdrop-blur-sm bg-
black/30 flex justify-center items-center'>
        <form onSubmit={onSubmitHandler} className='relative bg-white p-10 rounded-xl
text-slate-500'>
            <h1 className='text-center text-2xl text-neutral-700 font-medium'>Recruiter
{state}</h1>
            <p className='text-sm'>Welcome back! Please sign in to continue </p>
            {state === "Sign Up" && isTextDataSubmited
                ? <>

                    <div className='flex items-center gap-4 my-10'>
                        <label htmlFor="image">
                            <img className='w-16 rounded-full' src={image ?
URL.createObjectURL(image) : assets.upload_area} alt="" />
                            <input onChange={e => setImage(e.target.files[0])} type="file"
id='image' hidden />
                        </label>
                        <p>Upload Company <br /> logo</p>
                    </div>

                </>
                : <>

                    {state !== 'Login' && (
                        <div className='border px-4 py-2 flex items-center gap-2 rounded-full mt-
```

```
5'>
                        <img src={assets.person_icon} alt="" />
                        <input className='outline-none text-sm' onChange={e =>
setName(e.target.value)} value={name} type="text" placeholder='Company Name' required
/>
                    </div>
                )}

                <div className='border px-4 py-2 flex items-center gap-2 rounded-full mt-
5'>
                    <img src={assets.email_icon} alt="" />
                    <input className='outline-none text-sm' onChange={e =>
setEmail(e.target.value)} value={email} type="email" placeholder='Email Id' required />
                </div>

                <div className='border px-4 py-2 flex items-center gap-2 rounded-full mt-
5'>
                    <img src={assets.lock_icon} alt="" />
                    <input className='outline-none text-sm' onChange={e =>
setPassword(e.target.value)} value={password} type="password" placeholder='Password'
required />
                </div>


            </>}

        {state === "Login" && <p className='text-sm text-blue-600 mt-4 cursor-
pointer'>Forgot password?</p>}

        <button type='submit' className='bg-blue-600 w-full text-white py-2 rounded-full
mt-4'>
            {state === 'Login' ? 'login' : isTextDataSubmited ? 'create account' : 'next'}
        </button>

        {
            state === 'Login'
                ? <p className='mt-5 text-center'>Don't have an account? <span
className='text-blue-600 cursor-pointer' onClick={() => setState("Sign Up")}>Sign
Up</span></p>
                : <p className='mt-5 text-center'>Already have an account? <span
className='text-blue-600 cursor-pointer' onClick={() =>
setState("Login")}>Login</span></p>
        }

        <img onClick={e => setShowRecruiterLogin(false)} className='absolute top-5
right-5 cursor-pointer' src={assets.cross_icon} alt="" />
```

```
          </form>
        </div>
    )
}

export default RecruiterLogin


#client/src/App.jsx:
import { useContext } from 'react'
import { Route, Routes } from 'react-router-dom'
import Home from './pages/Home'
import ApplyJob from './pages/ApplyJob'
import Applications from './pages/Applications'
import RecruiterLogin from './components/RecruiterLogin'
import { AppContext } from './context/AppContext'
import Dashboard from './pages/Dashboard'
import AddJob from './pages/AddJob'
import ManageJobs from './pages/ManageJobs'
import ViewApplications from './pages/ViewApplications'
import 'quill/dist/quill.snow.css'
import { ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const App = () => {

  const { showRecruiterLogin, companyToken } = useContext(AppContext)

  return (
    <div>
      {showRecruiterLogin && <RecruiterLogin />}
      <ToastContainer />
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/apply-job/:id' element={<ApplyJob />} />
        <Route path='/applications' element={<Applications />} />
        <Route path='/dashboard' element={<Dashboard />}>
          {
            companyToken ? <>
              <Route path='add-job' element={<AddJob />} />
              <Route path='manage-jobs' element={<ManageJobs />} />
              <Route path='view-applications' element={<ViewApplications />} />
            </> : null
          }
        </Route>
```

```
      </Routes>
    </div>
  )
}

export default App


#client/src/main.jsx

import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { BrowserRouter } from 'react-router-dom'
import { AppContextProvider } from './context/AppContext.jsx'
import { ClerkProvider } from '@clerk/clerk-react'

// Import your publishable key
const PUBLISHABLE_KEY = import.meta.env.VITE_CLERK_PUBLISHABLE_KEY

if (!PUBLISHABLE_KEY) {
  throw new Error("Missing Publishable Key")
}

createRoot(document.getElementById('root')).render(
  <ClerkProvider publishableKey={PUBLISHABLE_KEY} afterSignOutUrl="/">
    <BrowserRouter>
      <AppContextProvider>
        <App />
      </AppContextProvider>
    </BrowserRouter>
  </ClerkProvider>,
)


SERVER:

#server/config/cloudinary.js:

import { v2 as cloudinary } from 'cloudinary'
const connectCloudinary = async () => {

    cloudinary.config({
        cloud_name: process.env.CLOUDINARY_NAME,
        api_key: process.env.CLOUDINARY_API_KEY,
        api_secret: process.env.CLOUDINARY_SECRET_KEY
```

```
    })

}
export default connectCloudinary

#server/config/db.js
import mongoose from "mongoose";

// Function to connect to the MongoDB database
const connectDB = async () => {

    mongoose.connection.on('connected', () => console.log('Database Connected'))

    await mongoose.connect(`${process.env.MONGODB_URI}/job-portal`)

}

export default connectDB

#server/config/multer.js
import multer from "multer";

const storage = multer.diskStorage({})

const upload = multer({ storage })

export default upload


#server/controllers/companyController.js
import Company from "../models/Company.js";
import bcrypt from 'bcrypt'
import { v2 as cloudinary } from 'cloudinary'
import generateToken from "../utils/generateToken.js";
import Job from "../models/Job.js";
import JobApplication from "../models/JobApplication.js";

// Register a new company
export const registerCompany = async (req, res) => {

    const { name, email, password } = req.body

    const imageFile = req.file;

    if (!name || !email || !password || !imageFile) {
        return res.json({ success: false, message: "Missing Details" })
```

```
    }

    try {

        const companyExists = await Company.findOne({ email })

        if (companyExists) {
            return res.json({ success: false, message: 'Company already registered' })
        }

        const salt = await bcrypt.genSalt(10)
        const hashPassword = await bcrypt.hash(password, salt)

        const imageUpload = await cloudinary.uploader.upload(imageFile.path)

        const company = await Company.create({
            name,
            email,
            password: hashPassword,
            image: imageUpload.secure_url
        })

        res.json({
            success: true,
            company: {
                _id: company._id,
                name: company.name,
                email: company.email,
                image: company.image
            },
            token: generateToken(company._id)
        })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}

// Login Company
export const loginCompany = async (req, res) => {

    const { email, password } = req.body

    try {

        const company = await Company.findOne({ email })
```

```
        if (await bcrypt.compare(password, company.password)) {

            res.json({
                success: true,
                company: {
                    _id: company._id,
                    name: company.name,
                    email: company.email,
                    image: company.image
                },
                token: generateToken(company._id)
            })

        }
        else {
            res.json({ success: false, message: 'Invalid email or password' })
        }

    } catch (error) {
        res.json({ success: false, message: error.message })
    }

}

// Get Company Data
export const getCompanyData = async (req, res) => {

    try {

        const company = req.company

        res.json({ success: true, company })

    } catch (error) {
        res.json({
            success: false, message: error.message
        })
    }

}

// Post New Job
export const postJob = async (req, res) => {

    const { title, description, location, salary, level, category } = req.body
```

```
    const companyId = req.company._id

    try {

        const newJob = new Job({
            title,
            description,
            location,
            salary,
            companyId,
            date: Date.now(),
            level,
            category
        })

        await newJob.save()

        res.json({ success: true, newJob })

    } catch (error) {

        res.json({ success: false, message: error.message })

    }


}

// Get Company Job Applicants
export const getCompanyJobApplicants = async (req, res) => {
    try {

        const companyId = req.company._id

        // Find job applications for the user and populate related data
        const applications = await JobApplication.find({ companyId })
            .populate('userId', 'name image resume')
            .populate('jobId', 'title location category level salary')
            .exec()

        return res.json({ success: true, applications })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
```

```
}

// Get Company Posted Jobs
export const getCompanyPostedJobs = async (req, res) => {
    try {

        const companyId = req.company._id

        const jobs = await Job.find({ companyId })

        // Adding No. of applicants info in data
        const jobsData = await Promise.all(jobs.map(async (job) => {
            const applicants = await JobApplication.find({ jobId: job._id });
            return { ...job.toObject(), applicants: applicants.length }
        }))

        res.json({ success: true, jobsData })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}

// Change Job Application Status
export const ChangeJobApplicationsStatus = async (req, res) => {

    try {

        const { id, status } = req.body

        // Find Job application and update status
        await JobApplication.findOneAndUpdate({ _id: id }, { status })

        res.json({ success: true, message: 'Status Changed' })

    } catch (error) {

        res.json({ success: false, message: error.message })

    }
}

// Change Job Visiblity
export const changeVisiblity = async (req, res) => {
    try {
```

```
        const { id } = req.body

        const companyId = req.company._id

        const job = await Job.findById(id)

        if (companyId.toString() === job.companyId.toString()) {
            job.visible = !job.visible
        }

        await job.save()

        res.json({ success: true, job })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}


#server/controllers/jobController.js
import Job from "../models/Job.js"



// Get All Jobs
export const getJobs = async (req, res) => {
    try {

        const jobs = await Job.find({ visible: true })
            .populate({ path: 'companyId', select: '-password' })

        res.json({ success: true, jobs })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}

// Get Single Job Using JobID
export const getJobById = async (req, res) => {
    try {

        const { id } = req.params

        const job = await Job.findById(id)
```

```
            .populate({
                path: 'companyId',
                select: '-password'
            })

        if (!job) {
            return res.json({
                success: false,
                message: 'Job not found'
            })
        }

        res.json({
            success: true,
            job
        })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}


#server/controllers/webhooks.js
import { Webhook } from "svix";
import User from "../models/User.js";

// API Controller Function to Manage Clerk User with database
export const clerkWebhooks = async (req, res) => {
    try {

        // Create a Svix instance with clerk webhook secret.
        const whook = new Webhook(process.env.CLERK_WEBHOOK_SECRET)

        // Verifying Headers
        await whook.verify(JSON.stringify(req.body), {
            "svix-id": req.headers["svix-id"],
            "svix-timestamp": req.headers["svix-timestamp"],
            "svix-signature": req.headers["svix-signature"]
        })

        // Getting Data from request body
        const { data, type } = req.body

        // Switch Cases for differernt Events
        switch (type) {
```

```
        case 'user.created': {

            const userData = {
                _id: data.id,
                email: data.email_addresses[0].email_address,
                name: data.first_name + " " + data.last_name,
                image: data.image_url,
                resume: "
            }
            await User.create(userData)
            res.json({})
            break;
        }

        case 'user.updated': {
            const userData = {
                email: data.email_addresses[0].email_address,
                name: data.first_name + " " + data.last_name,
                image: data.image_url,
            }
            await User.findByIdAndUpdate(data.id, userData)
            res.json({})
            break;
        }

        case 'user.deleted': {
            await User.findByIdAndDelete(data.id)
            res.json({})
            break;
        }
        default:
            break;
    }

  } catch (error) {
    res.json({ success: false, message: error.message })
  }
}


#server/controllers/companyController.js
import Company from "../models/Company.js";
import bcrypt from 'bcrypt'
import { v2 as cloudinary } from 'cloudinary'
import generateToken from "../utils/generateToken.js";
import Job from "../models/Job.js";
```

```
import JobApplication from "../models/JobApplication.js";

// Register a new company
export const registerCompany = async (req, res) => {

    const { name, email, password } = req.body

    const imageFile = req.file;

    if (!name || !email || !password || !imageFile) {
        return res.json({ success: false, message: "Missing Details" })
    }

    try {

        const companyExists = await Company.findOne({ email })

        if (companyExists) {
            return res.json({ success: false, message: 'Company already registered' })
        }

        const salt = await bcrypt.genSalt(10)
        const hashPassword = await bcrypt.hash(password, salt)

        const imageUpload = await cloudinary.uploader.upload(imageFile.path)

        const company = await Company.create({
            name,
            email,
            password: hashPassword,
            image: imageUpload.secure_url
        })

        res.json({
            success: true,
            company: {
                _id: company._id,
                name: company.name,
                email: company.email,
                image: company.image
            },
            token: generateToken(company._id)
        })

    } catch (error) {
        res.json({ success: false, message: error.message })
```

```
    }
}

// Login Company
export const loginCompany = async (req, res) => {

    const { email, password } = req.body

    try {

        const company = await Company.findOne({ email })

        if (await bcrypt.compare(password, company.password)) {

            res.json({
                success: true,
                company: {
                    _id: company._id,
                    name: company.name,
                    email: company.email,
                    image: company.image
                },
                token: generateToken(company._id)
            })

        }
        else {
            res.json({ success: false, message: 'Invalid email or password' })
        }

    } catch (error) {
        res.json({ success: false, message: error.message })
    }

}

// Get Company Data
export const getCompanyData = async (req, res) => {

    try {

        const company = req.company

        res.json({ success: true, company })

    } catch (error) {
```

```
        res.json({
            success: false, message: error.message
        })
    }

}

// Post New Job
export const postJob = async (req, res) => {

    const { title, description, location, salary, level, category } = req.body

    const companyId = req.company._id

    try {

        const newJob = new Job({
            title,
            description,
            location,
            salary,
            companyId,
            date: Date.now(),
            level,
            category
        })

        await newJob.save()

        res.json({ success: true, newJob })

    } catch (error) {

        res.json({ success: false, message: error.message })

    }


}

// Get Company Job Applicants
export const getCompanyJobApplicants = async (req, res) => {
    try {

        const companyId = req.company._id
```

```
        // Find job applications for the user and populate related data
        const applications = await JobApplication.find({ companyId })
            .populate('userId', 'name image resume')
            .populate('jobId', 'title location category level salary')
            .exec()

        return res.json({ success: true, applications })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}

// Get Company Posted Jobs
export const getCompanyPostedJobs = async (req, res) => {
    try {

        const companyId = req.company._id

        const jobs = await Job.find({ companyId })

        // Adding No. of applicants info in data
        const jobsData = await Promise.all(jobs.map(async (job) => {
            const applicants = await JobApplication.find({ jobId: job._id });
            return { ...job.toObject(), applicants: applicants.length }
        }))

        res.json({ success: true, jobsData })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}

// Change Job Application Status
export const ChangeJobApplicationsStatus = async (req, res) => {

    try {

        const { id, status } = req.body

        // Find Job application and update status
        await JobApplication.findOneAndUpdate({ _id: id }, { status })

        res.json({ success: true, message: 'Status Changed' })
```

```
    } catch (error) {

        res.json({ success: false, message: error.message })

    }
}

// Change Job Visiblity
export const changeVisiblity = async (req, res) => {
    try {

        const { id } = req.body

        const companyId = req.company._id

        const job = await Job.findById(id)

        if (companyId.toString() === job.companyId.toString()) {
            job.visible = !job.visible
        }

        await job.save()

        res.json({ success: true, job })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
}


#server/controllers/userController.js
import Job from "../models/Job.js"
import JobApplication from "../models/JobApplication.js"
import User from "../models/User.js"
import { v2 as cloudinary } from "cloudinary"

// Get User Data
export const getUserData = async (req, res) => {

    const userId = req.auth.userId

    try {

        const user = await User.findById(userId)
```

```
        if (!user) {
            return res.json({ success: false, message: 'User Not Found' })
        }

        res.json({ success: true, user })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }

}


// Apply For Job
export const applyForJob = async (req, res) => {

    const { jobId } = req.body

    const userId = req.auth.userId

    try {

        const isAlreadyApplied = await JobApplication.find({ jobId, userId })

        if (isAlreadyApplied.length > 0) {
            return res.json({ success: false, message: 'Already Applied' })
        }

        const jobData = await Job.findById(jobId)

        if (!jobData) {
            return res.json({ success: false, message: 'Job Not Found' })
        }

        await JobApplication.create({
            companyId: jobData.companyId,
            userId,
            jobId,
            date: Date.now()
        })

        res.json({ success: true, message: 'Applied Successfully' })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }
```

```
}

// Get User Applied Applications Data
export const getUserJobApplications = async (req, res) => {

    try {

        const userId = req.auth.userId

        const applications = await JobApplication.find({ userId })
            .populate('companyId', 'name email image')
            .populate('jobId', 'title description location category level salary')
            .exec()

        if (!applications) {
            return res.json({ success: false, message: 'No job applications found for this user.' })
        }

        return res.json({ success: true, applications })

    } catch (error) {
        res.json({ success: false, message: error.message })
    }

}

// Update User Resume
export const updateUserResume = async (req, res) => {
    try {

        const userId = req.auth.userId

        const resumeFile = req.file

        const userData = await User.findById(userId)

        if (resumeFile) {
            const resumeUpload = await cloudinary.uploader.upload(resumeFile.path)
            userData.resume = resumeUpload.secure_url
        }

        await userData.save()

        return res.json({ success: true, message: 'Resume Updated' })
```

```
    } catch (error) {

        res.json({ success: false, message: error.message })

    }
}



#server/middleware/authMiddleware.js
import jwt from 'jsonwebtoken'
import Company from '../models/Company.js'

// Middleware ( Protect Company Routes )
export const protectCompany = async (req,res,next) => {

    // Getting Token Froms Headers
    const token = req.headers.token


    if (!token) {
        return res.json({ success:false, message:'Not authorized, Login Again'})
    }

    try {

        const decoded = jwt.verify(token, process.env.JWT_SECRET)

        req.company = await Company.findById(decoded.id).select('-password')

        next()

    } catch (error) {
        res.json({success:false, message: error.message})
    }

}


#server/routes/companyRoutes.js
import express from 'express'
import { ChangeJobApplicationsStatus, changeVisiblity, getCompanyData,
getCompanyJobApplicants, getCompanyPostedJobs, loginCompany, postJob,
registerCompany } from '../controllers/companyController.js'
import upload from '../config/multer.js'
import { protectCompany } from '../middleware/authMiddleware.js'
```

```
const router = express.Router()

// Register a company
router.post('/register', upload.single('image'), registerCompany)

// Company login
router.post('/login', loginCompany)

// Get company data
router.get('/company', protectCompany, getCompanyData)

// Post a job
router.post('/post-job', protectCompany, postJob)

// Get Applicants Data of Company
router.get('/applicants', protectCompany, getCompanyJobApplicants)

// Get  Company Job List
router.get('/list-jobs', protectCompany, getCompanyPostedJobs)

// Change Applcations Status
router.post('/change-status', protectCompany, ChangeJobApplicationsStatus)

// Change Applcations Visiblity
router.post('/change-visiblity', protectCompany, changeVisiblity)

export default router


#server/routes/jobRoutes.js
import express from 'express'
import { getJobById, getJobs } from '../controllers/jobController.js';

const router = express.Router()

// Route to get all jobs data
router.get('/', getJobs)

// Route to get a single job by ID
router.get('/:id', getJobById)


export default router;
```

```
#server/routes/userRoutes.js
import express from 'express'
import { applyForJob, getUserData, getUserJobApplications, updateUserResume } from
'../controllers/userController.js'
import upload from '../config/multer.js'


const router = express.Router()

// Get user Data
router.get('/user', getUserData)

// Apply for a job
router.post('/apply', applyForJob)

// Get applied jobs data
router.get('/applications', getUserJobApplications)

// Update user profile (resume)
router.post('/update-resume', upload.single('resume'), updateUserResume)

export default router;


#server/utils/generateToken.js
import jwt from 'jsonwebtoken'

const generateToken = (id) => {
    return jwt.sign({ id }, process.env.JWT_SECRET, {
        expiresIn: '30d'
    })
}

export default generateToken




#server/server.js
import './config/instrument.js'
import express from 'express'
import cors from 'cors'
import 'dotenv/config'
import connectDB from './config/db.js'
import * as Sentry from "@sentry/node";
import { clerkWebhooks } from './controllers/webhooks.js'
```

```
import companyRoutes from './routes/companyRoutes.js'
import connectCloudinary from './config/cloudinary.js'
import jobRoutes from './routes/jobRoutes.js'
import userRoutes from './routes/userRoutes.js'
import { clerkMiddleware } from '@clerk/express'


// Initialize Express
const app = express()

// Connect to database
connectDB()
await connectCloudinary()

// Middlewares
app.use(cors())
app.use(express.json())
app.use(clerkMiddleware())

// Routes
app.get('/', (req, res) => res.send("API Working"))
app.get("/debug-sentry", function mainHandler(req, res) {
  throw new Error("My first Sentry error!");
});
app.post('/webhooks', clerkWebhooks)
app.use('/api/company', companyRoutes)
app.use('/api/jobs', jobRoutes)
app.use('/api/users', userRoutes)

// Port
const PORT = process.env.PORT || 5000

Sentry.setupExpressErrorHandler(app);

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
})
```